



Topic

Multi-layer Perceptron

CSE465: Pattern Recognition and Neural Network

Sec: 1

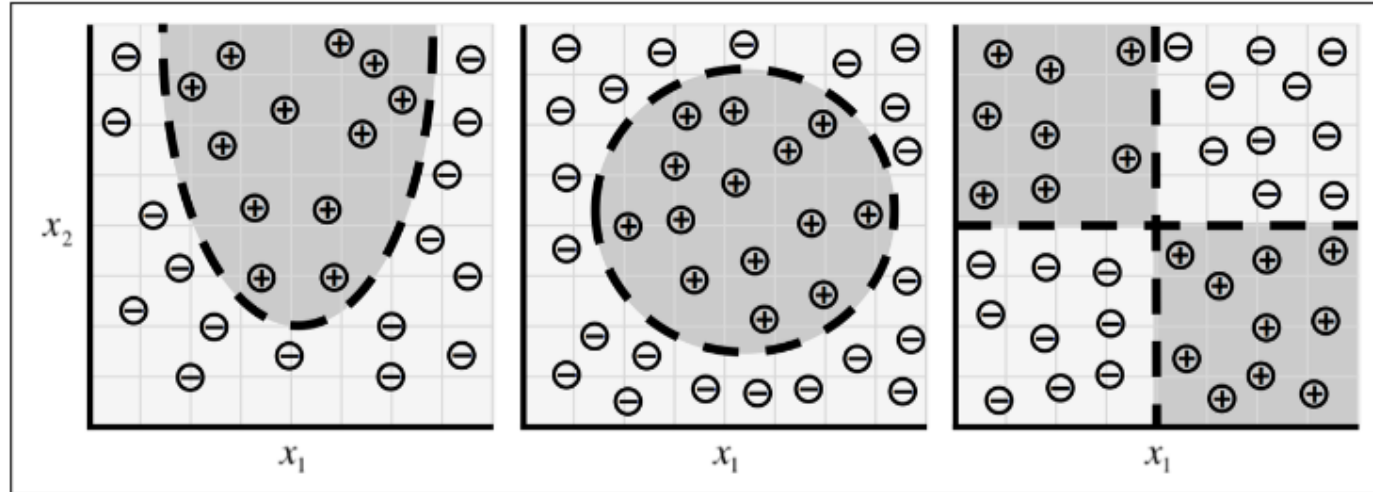
Summer 2025

Today's Topic

1. Multi layer perceptron
 - a) Problems with Perceptron
 - b) MLP Notation
 - c) Forward Propagation
 - d) Loss function
 - e) Gradient Descent
 - f) Backpropagation

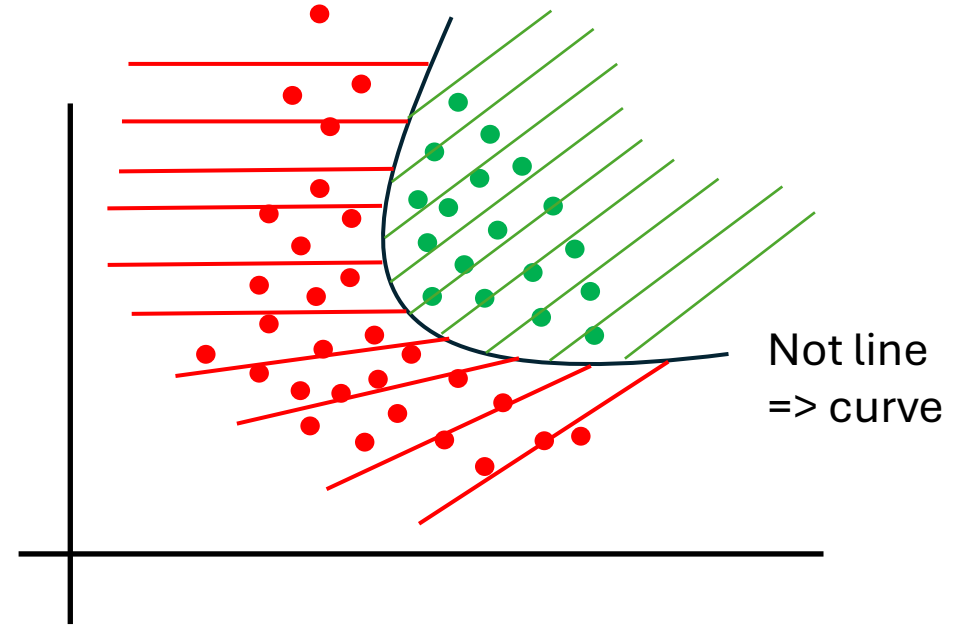
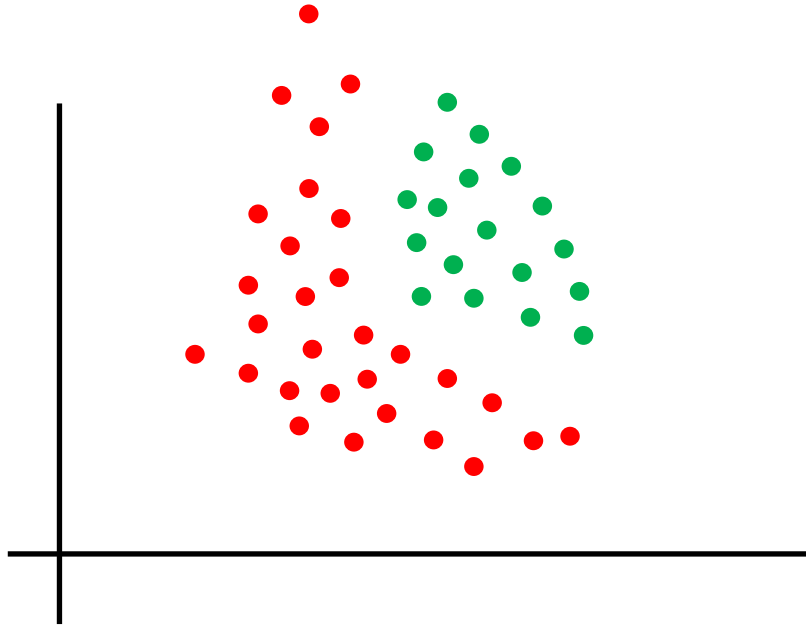
Limitation of a Perceptron

- Works only with linear or “sort-of” linear data

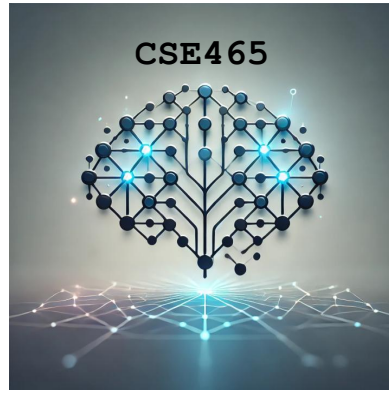


- Solution:
 - Multi-layer Perceptron (a.k.a. Artificial Neural Network)

The problem

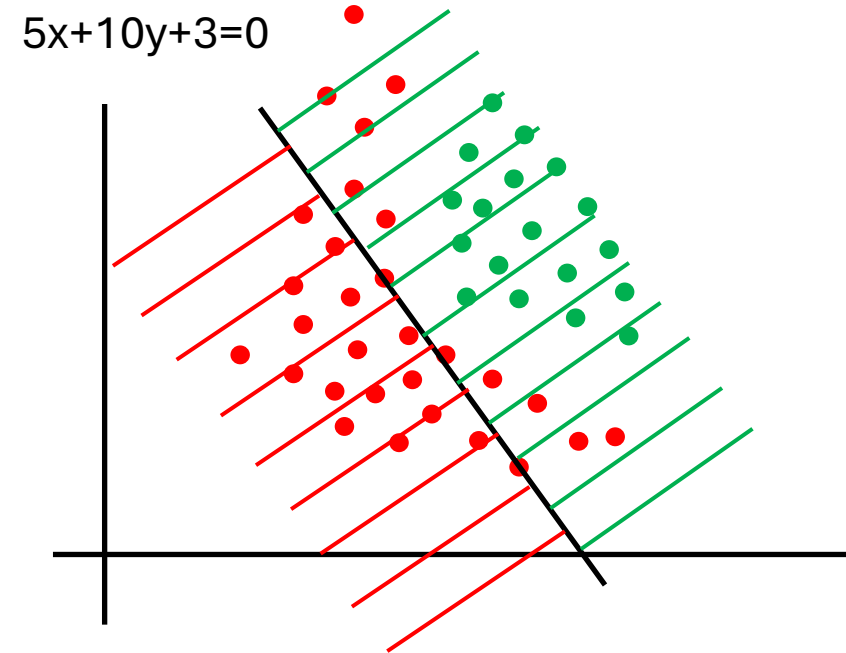
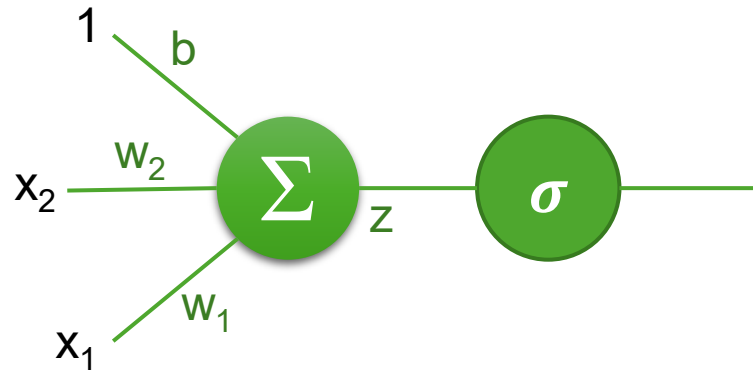


- Perceptron doesn't work on a dataset like this, as perceptron ends up with a line. We need a curve as the decision boundary.
- Challenge: we can only use Perceptron(s) to build such a system.



Emergence of multi-layer perceptron (MLP)

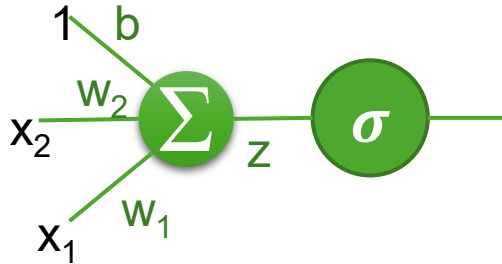
Perceptron with Sigmoid



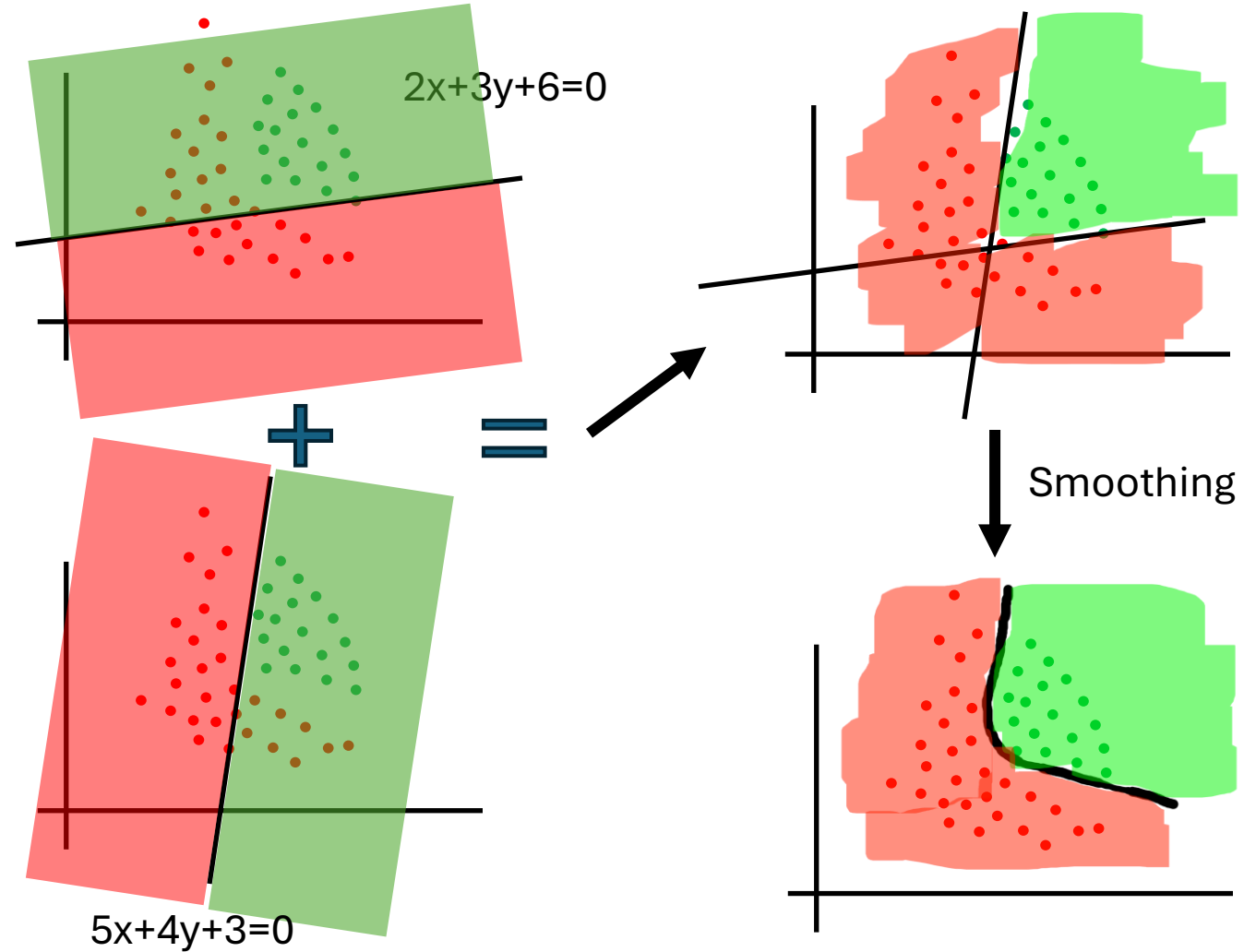
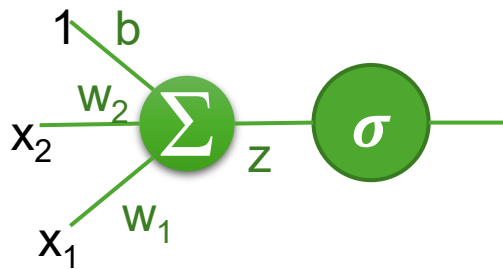
CGPA, x_1	IQ, x_2	z	$\sigma = \frac{1}{1 + e^{-z}}$
3.7	87	$5 \times 3.7 + 10 \times 87 + 3 = 891.5$	1

MLP Intuition

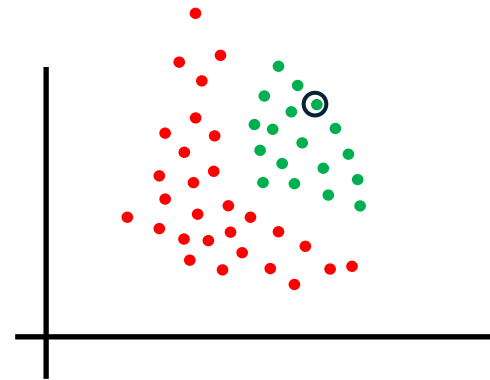
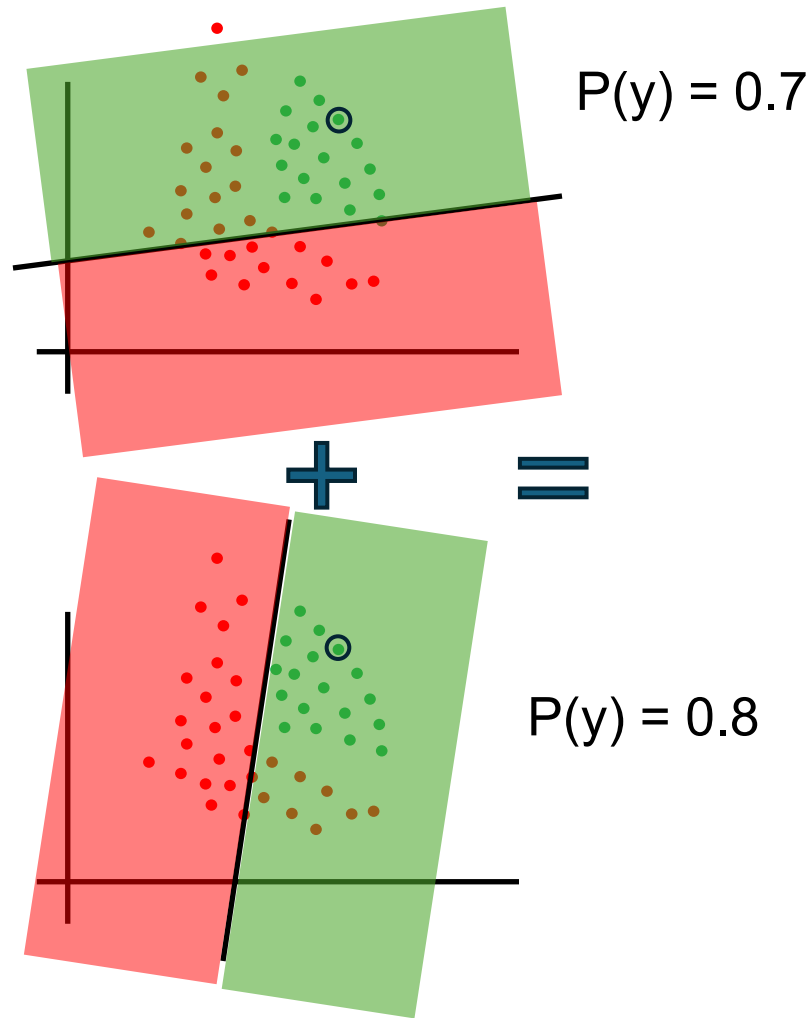
$$w_1 = 2, w_2 = 3, b = 6$$



$$w_1 = 5, w_2 = 4, b = 3$$



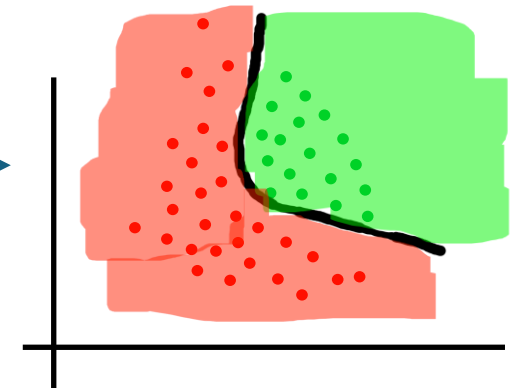
MLP: Mathematics



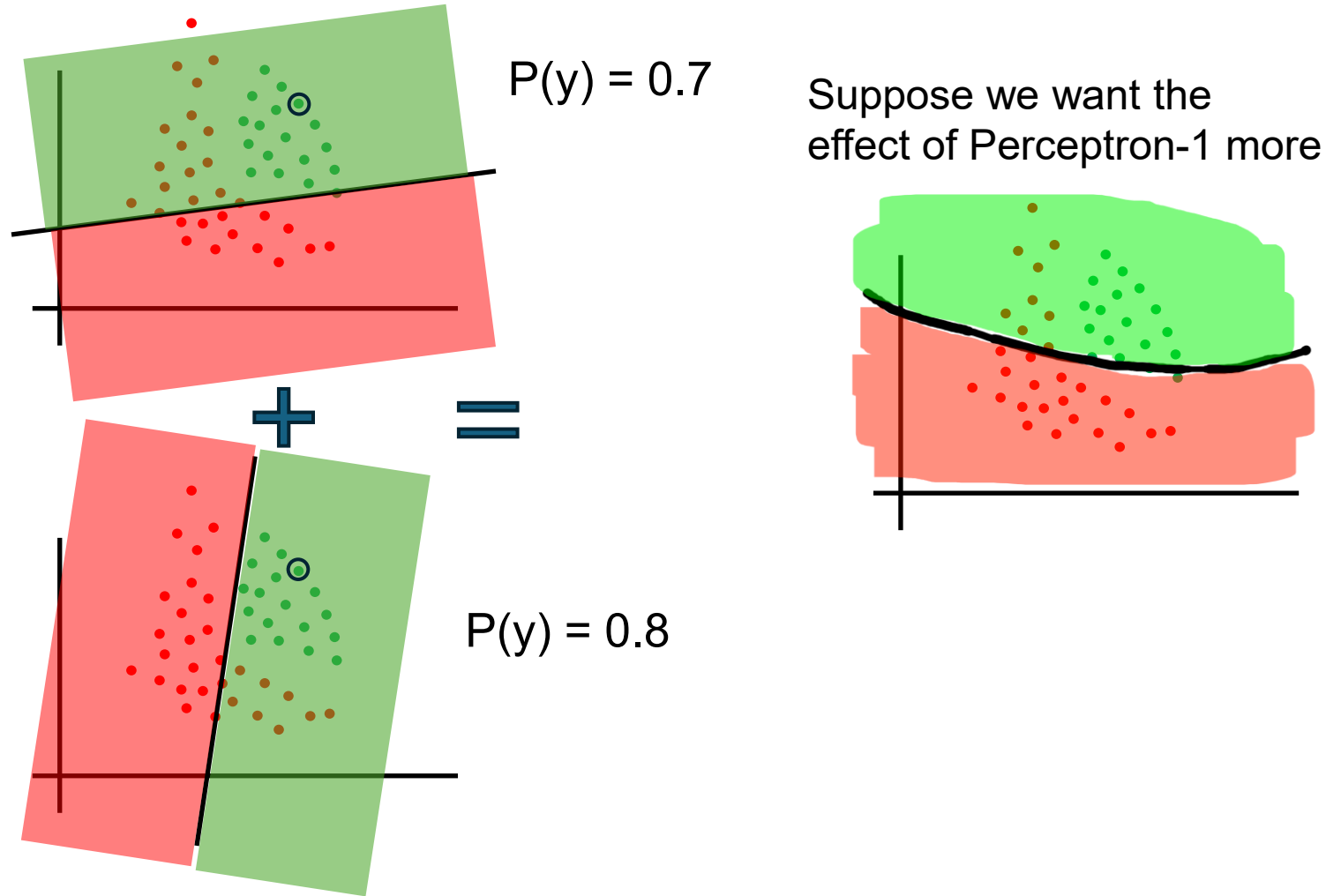
$$0.7 + 0.8 = 1.5 (!) \rightarrow \sigma$$
$$\frac{1}{1 + e^{-1.5}} = 0.82$$

Therefore,

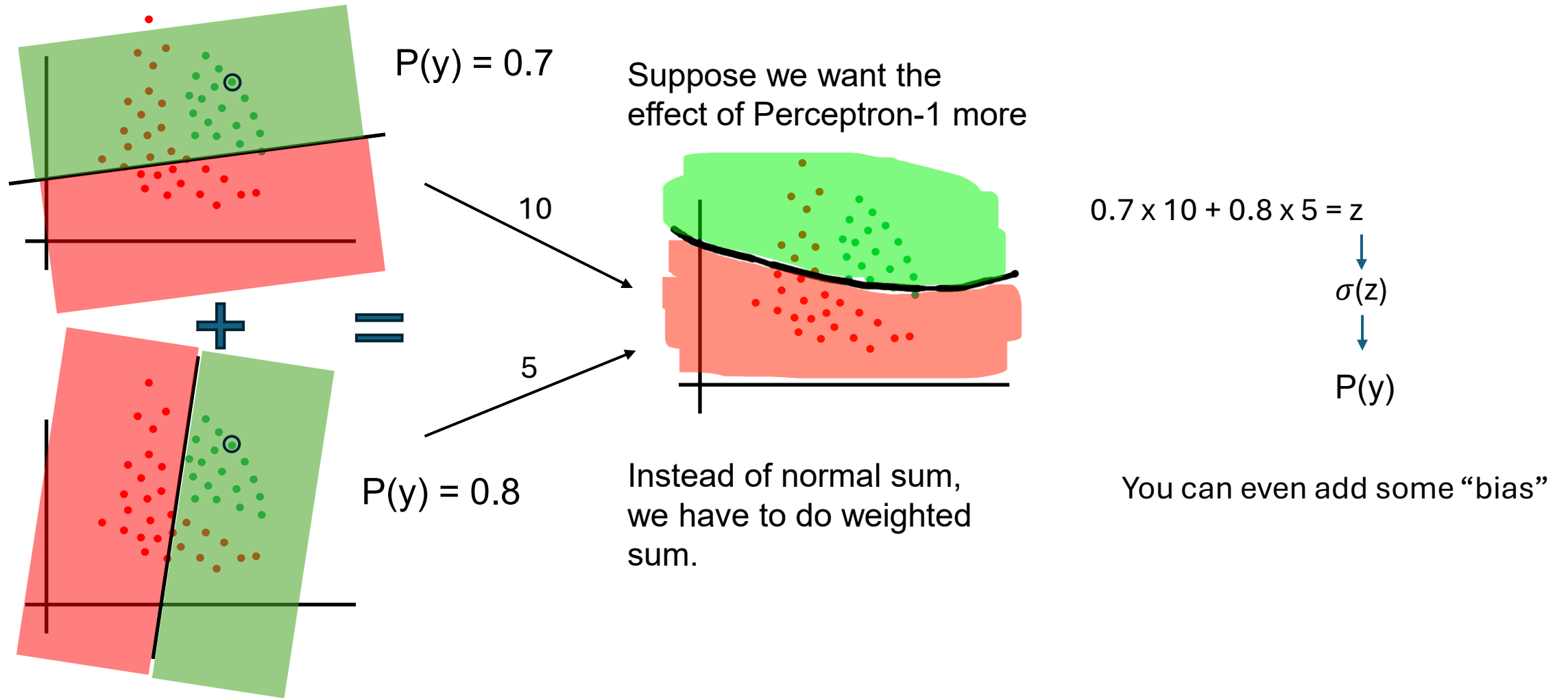
- First take addition of perceptrons.
- Then apply sigmoid as activation.



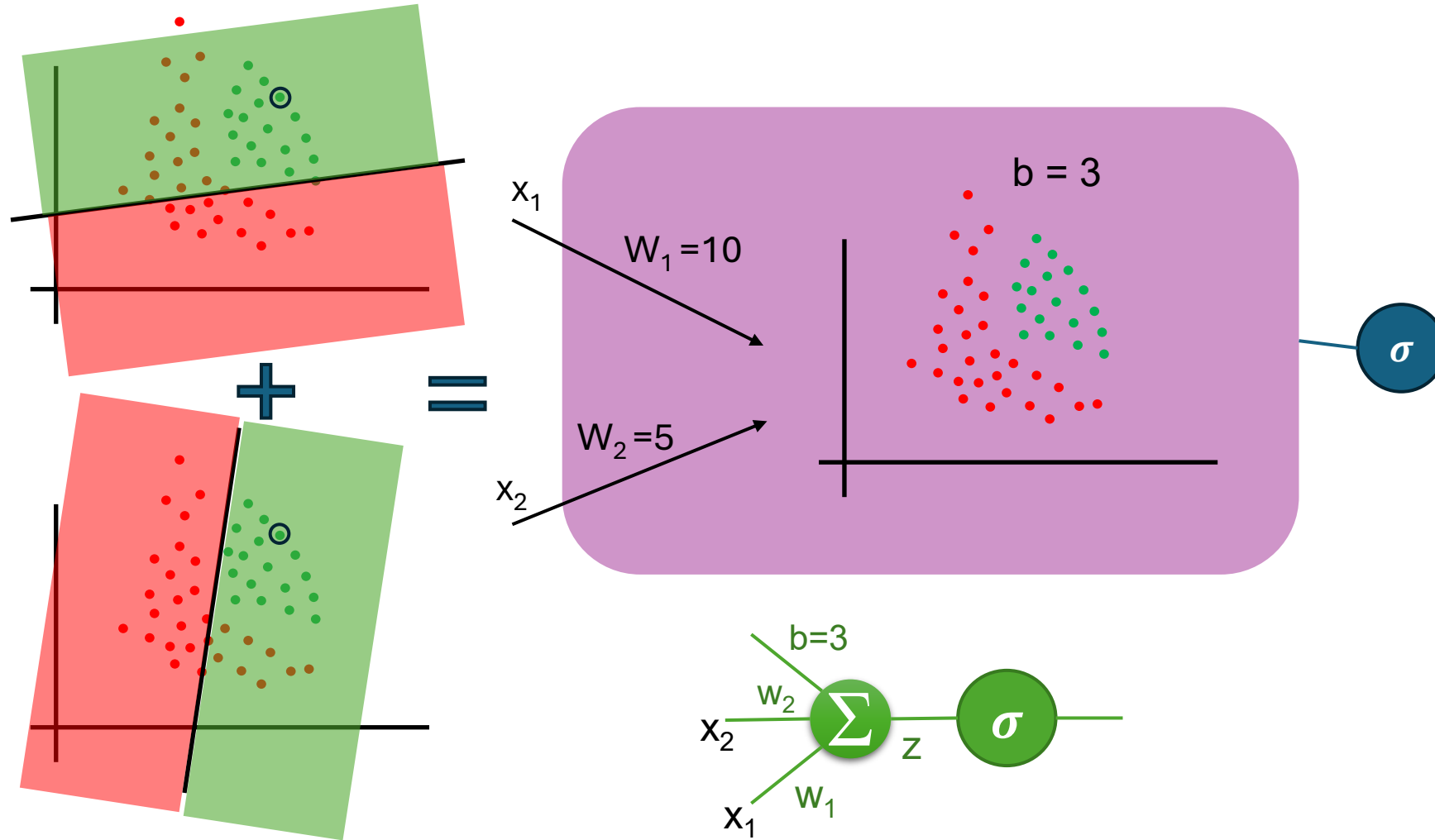
MLP: Mathematics (further enhancements)



MLP: Mathematics (further enhancements)



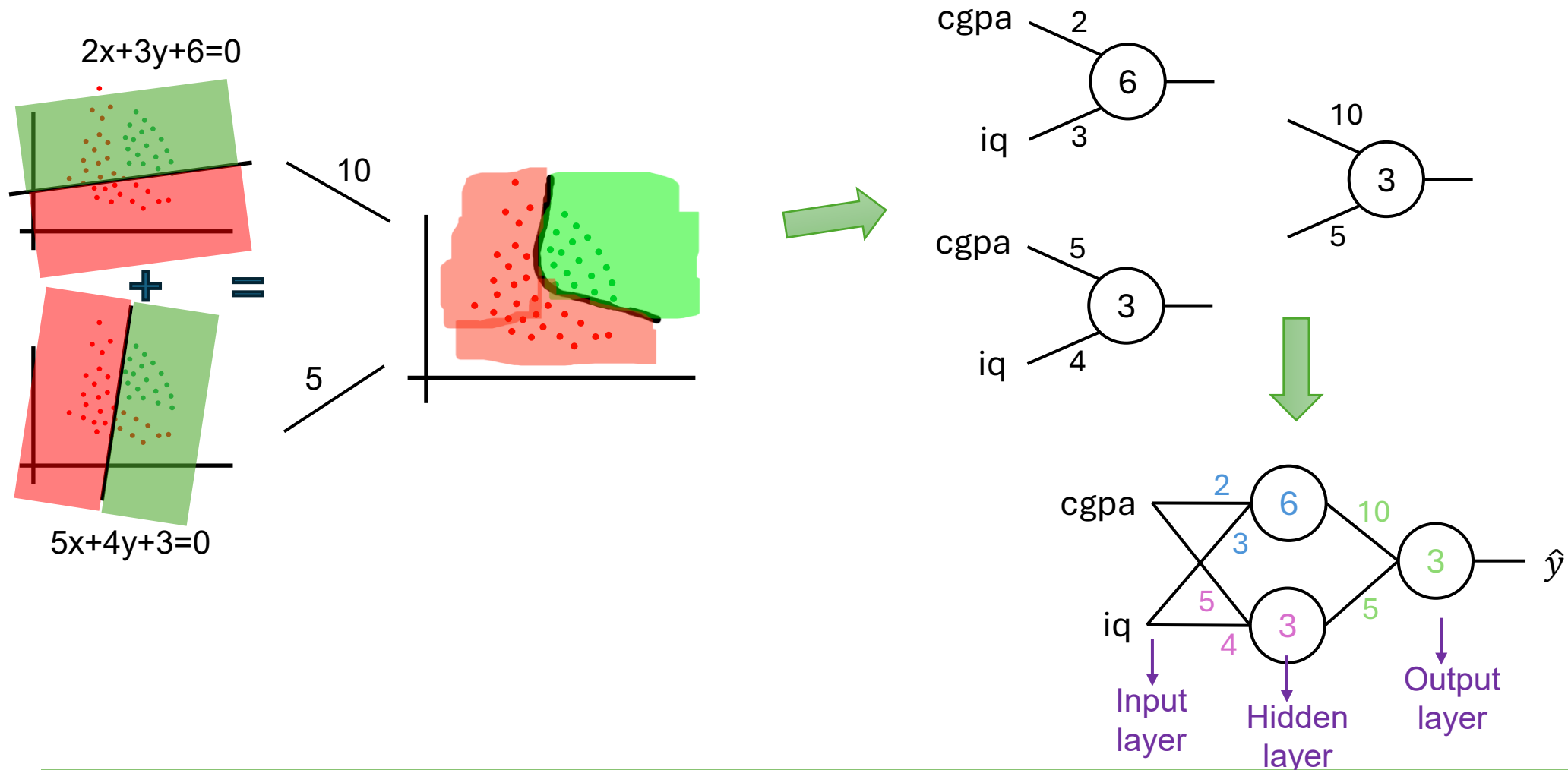
MLP: Mathematics (further enhancements)



So the entire thing works as a Perceptron, where the inputs are the outputs of the 2 perceptrons.

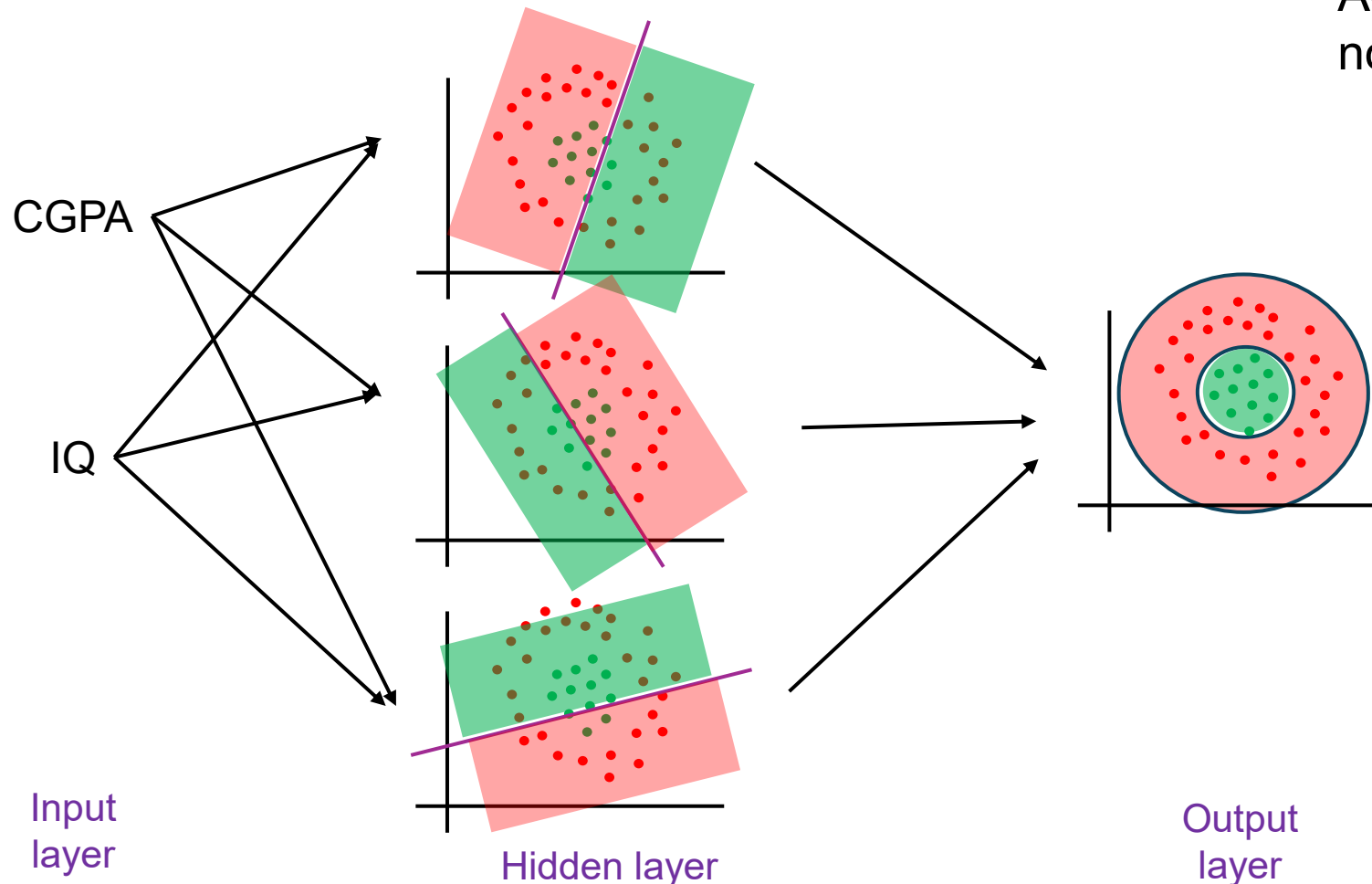
That is, it is a multi-layered perceptron

MLP: Formation



MLP: Formation with more flexibility

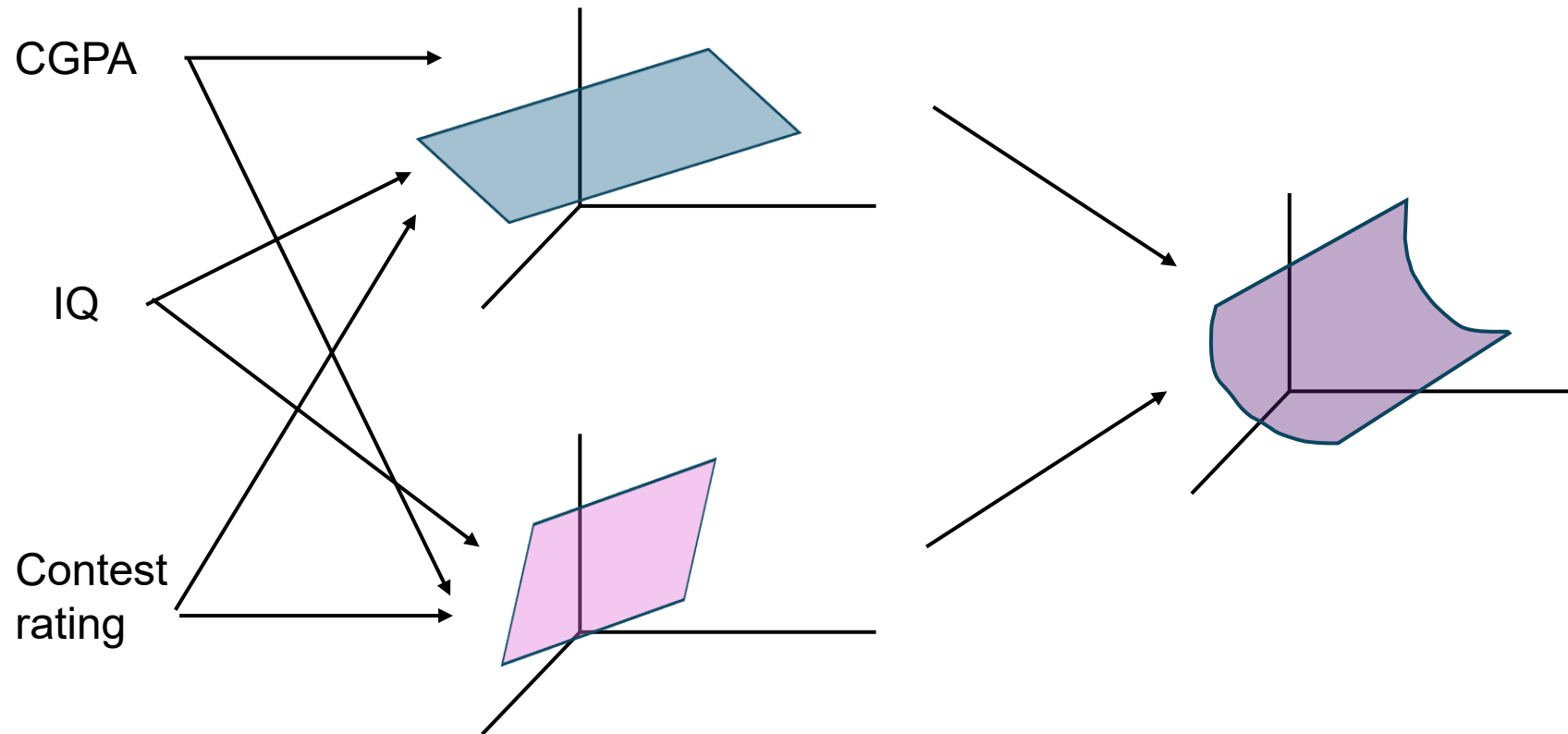
- Adding nodes in hidden layer



Additional and more complex non-linearity can be captured

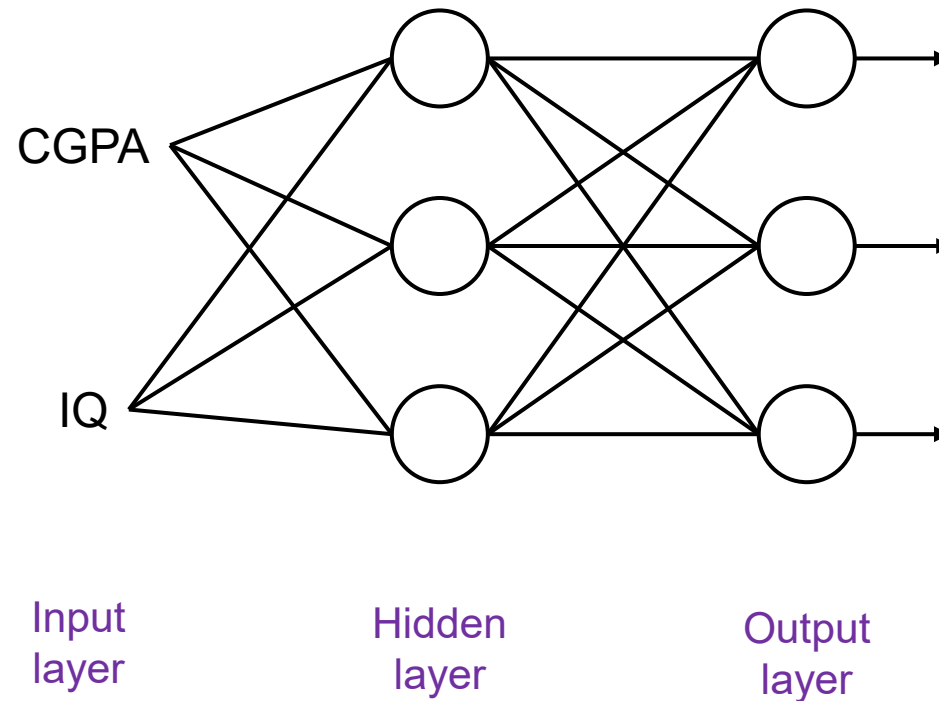
MLP: Formation with more flexibility

- Adding nodes in input layer (with extra features)



MLP: Formation with more flexibility

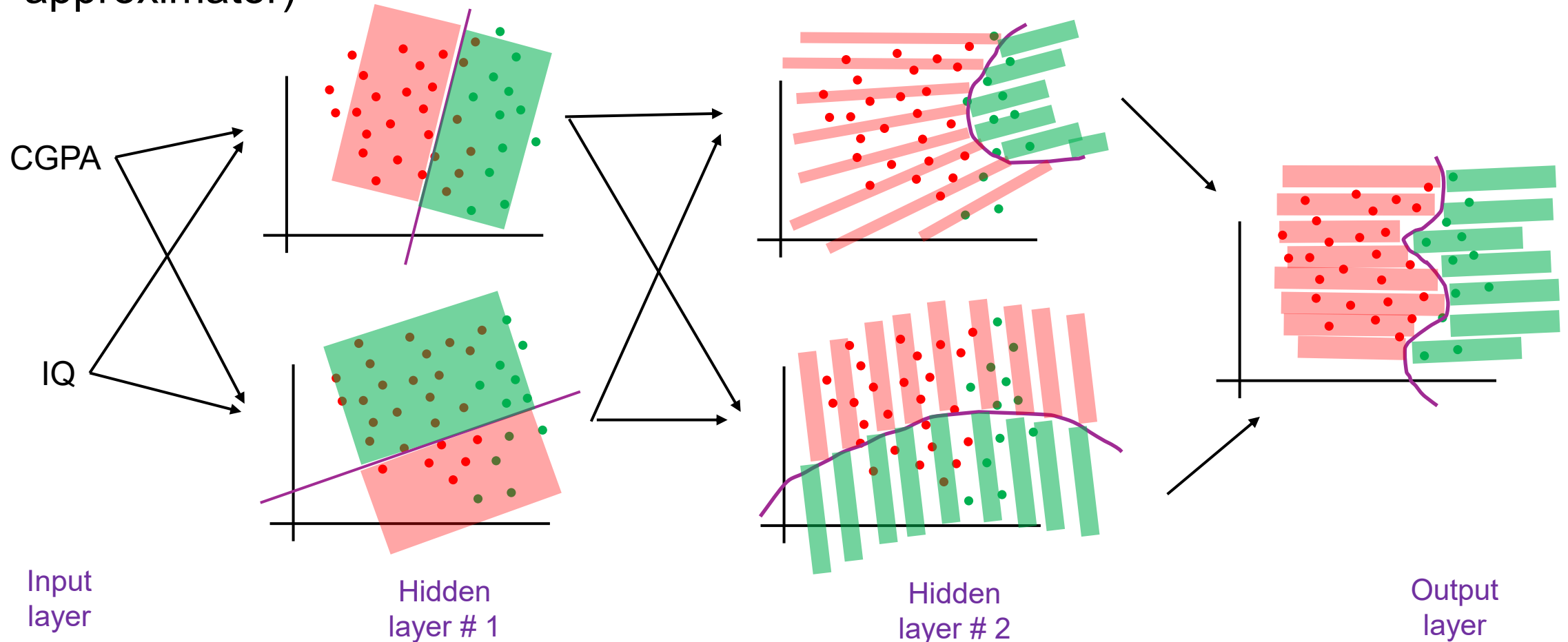
- Adding nodes in output layer

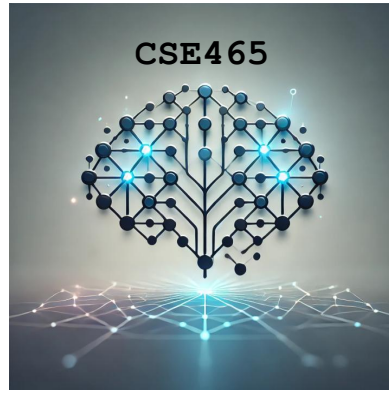


Used in multi-class classification.
E.g. Object classification

MLP: Formation with more flexibility

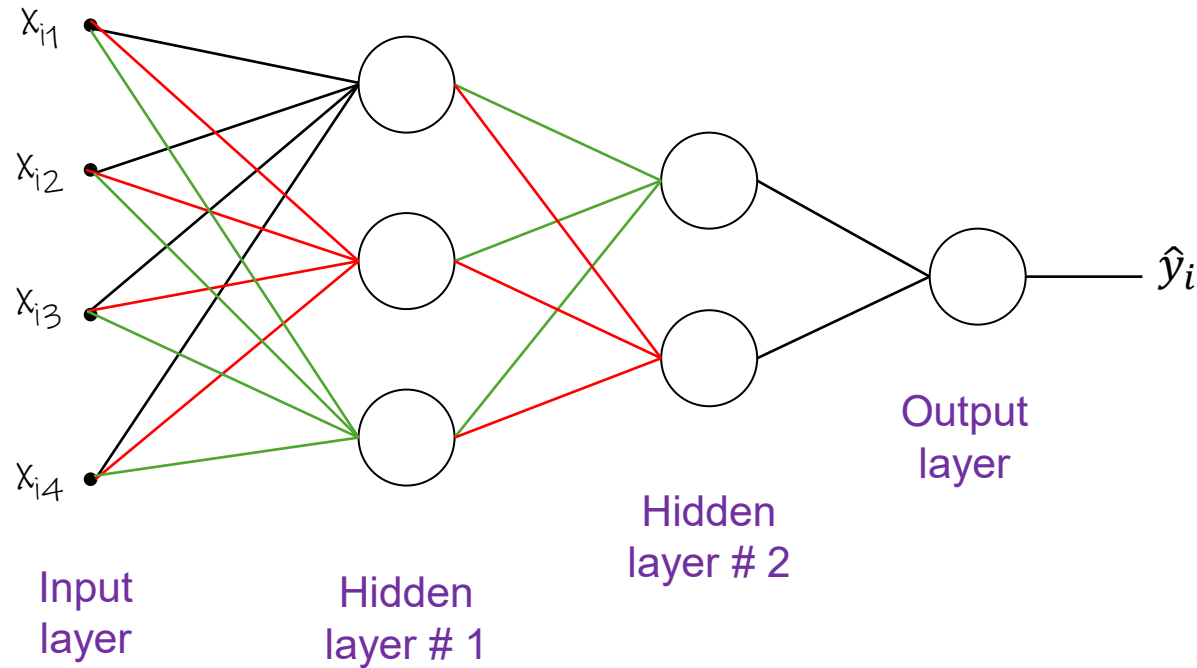
- Adding more hidden layers (Deep Neural Network, a.k.a. Universal Function approximator)





MLP/ANN Notation

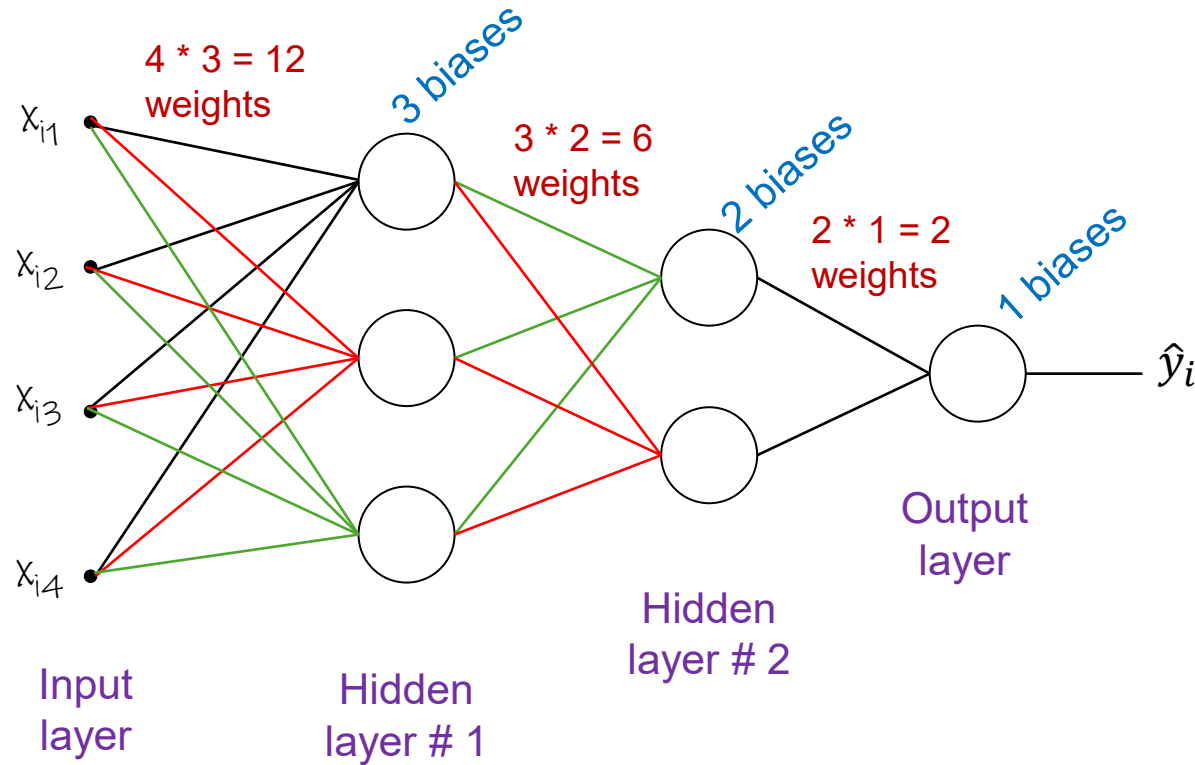
MLP Notation



Q1: What is the total number of trainable parameters for a neural network?

- Data $\rightarrow \{m \times n\}$
- $m \rightarrow$ No. of rows (samples)
- $n \rightarrow$ number of input features
- Each row is denoted by x_i

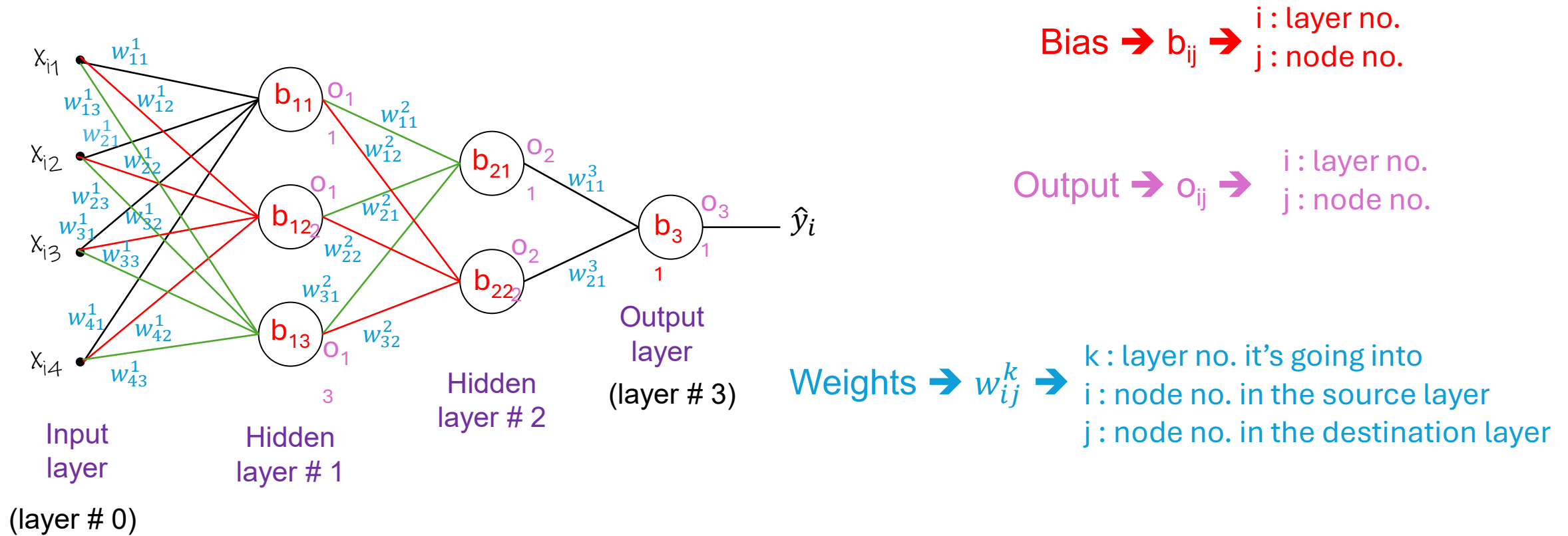
MLP Notation

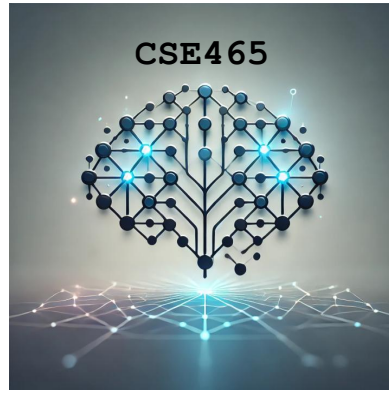


Q1: What is the total number of trainable parameters for a neural network?

$$\begin{array}{c} \text{Layer 1} \quad \text{Layer 2} \quad \text{Layer 3} \\ \text{Total parameters} = \underbrace{(12 + 3)}_{\text{Layer 1}} + \underbrace{(6 + 2)}_{\text{Layer 2}} + \underbrace{(2 + 1)}_{\text{Layer 3}} \\ = 26 \end{array}$$

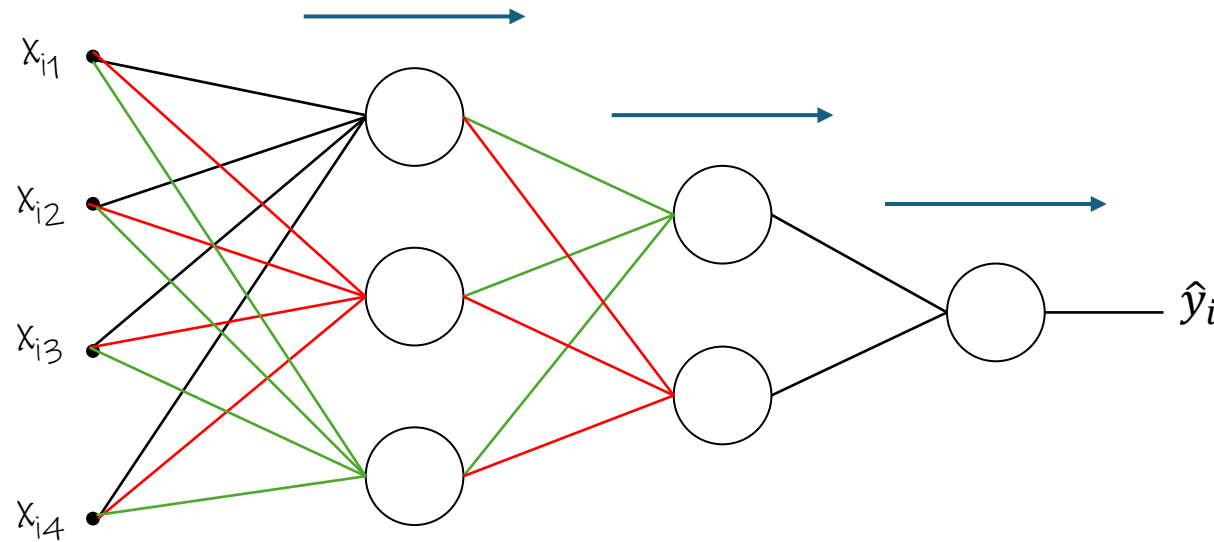
MLP Notation





Forward Propagation

Forward Propagation



Starting from the input, calculations are done layer-by-layer to finally calculate the prediction, $\hat{y}_i$

Forward Propagation: layer 1

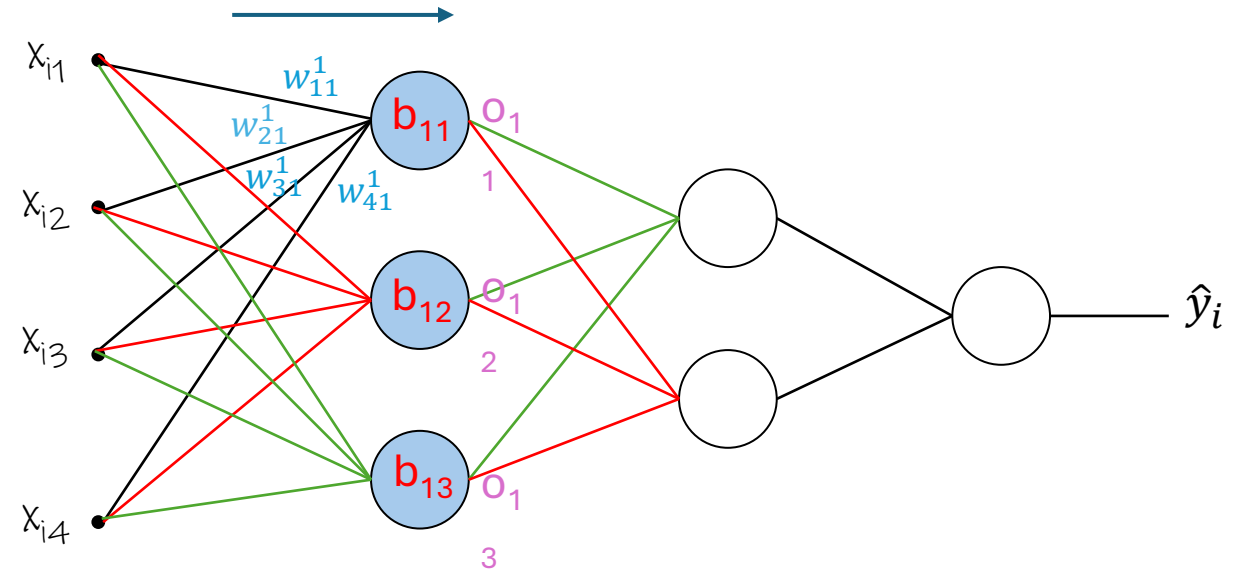
$$z_{11} = w_{11}^1 x_{i1} + w_{21}^1 x_{i2} + w_{31}^1 x_{i3} + w_{41}^1 x_{i4} + b_{11}$$
$$o_{11} = \sigma(z_{11})$$

$$z_{12} = w_{12}^1 x_{i1} + w_{22}^1 x_{i2} + w_{32}^1 x_{i3} + w_{42}^1 x_{i4} + b_{12}$$
$$o_{12} = \sigma(z_{12})$$

$$z_{13} = w_{13}^1 x_{i1} + w_{23}^1 x_{i2} + w_{33}^1 x_{i3} + w_{43}^1 x_{i4} + b_{13}$$
$$o_{13} = \sigma(z_{13})$$

Fortunately, this can be done easily using matrix calculation

$$\begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \\ w_{41}^1 & w_{42}^1 & w_{43}^1 \end{bmatrix}^T \cdot \begin{bmatrix} x_{i1} \\ x_{i2} \\ x_{i3} \\ x_{i4} \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix} = \begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix} \quad \longrightarrow \quad \sigma \left(\begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix} \right) = \begin{bmatrix} o_{11} \\ o_{12} \\ o_{13} \end{bmatrix}$$



Forward Propagation: layer 3 / Output layer

$$z_{21} = w_{11}^2 o_{11} + w_{21}^2 o_{12} + w_{31}^2 o_{13} + b_{21}$$
$$o_{21} = \sigma(z_{21})$$

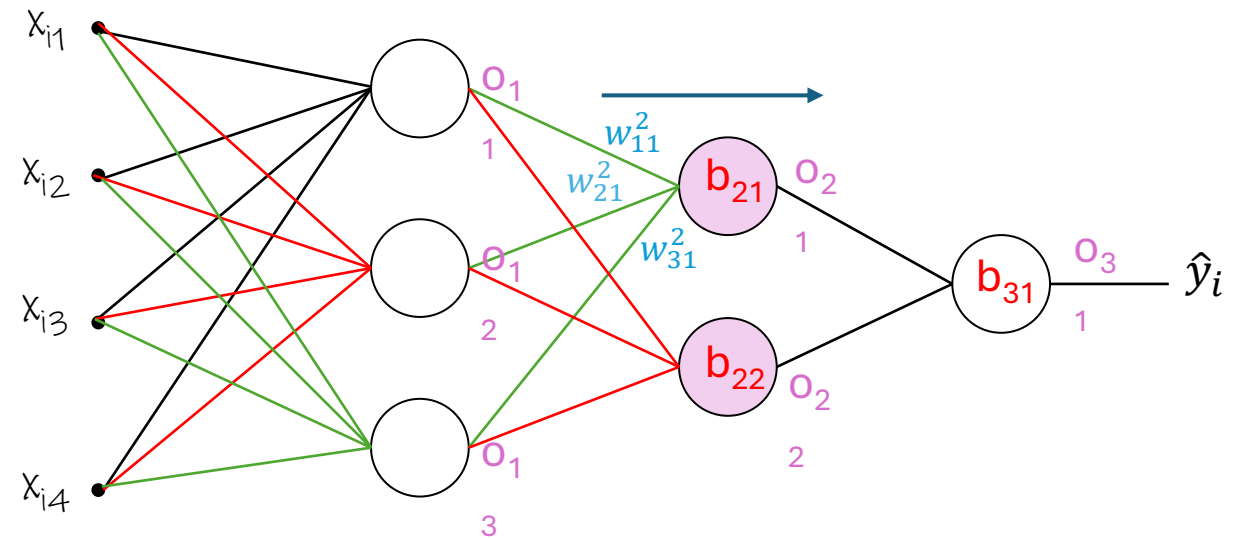
$$z_{22} = w_{12}^2 o_{11} + w_{22}^2 o_{12} + w_{32}^2 o_{13} + b_{22}$$
$$o_{22} = \sigma(z_{22})$$

Fortunately, this can be done easily using matrix calculation

$$\begin{bmatrix} w_{11}^2 & w_{12}^2 \\ w_{21}^2 & w_{22}^2 \\ w_{31}^2 & w_{32}^2 \end{bmatrix}^T \cdot \begin{bmatrix} o_{11} \\ o_{12} \\ o_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix} = \begin{bmatrix} z_{21} \\ z_{22} \end{bmatrix}$$

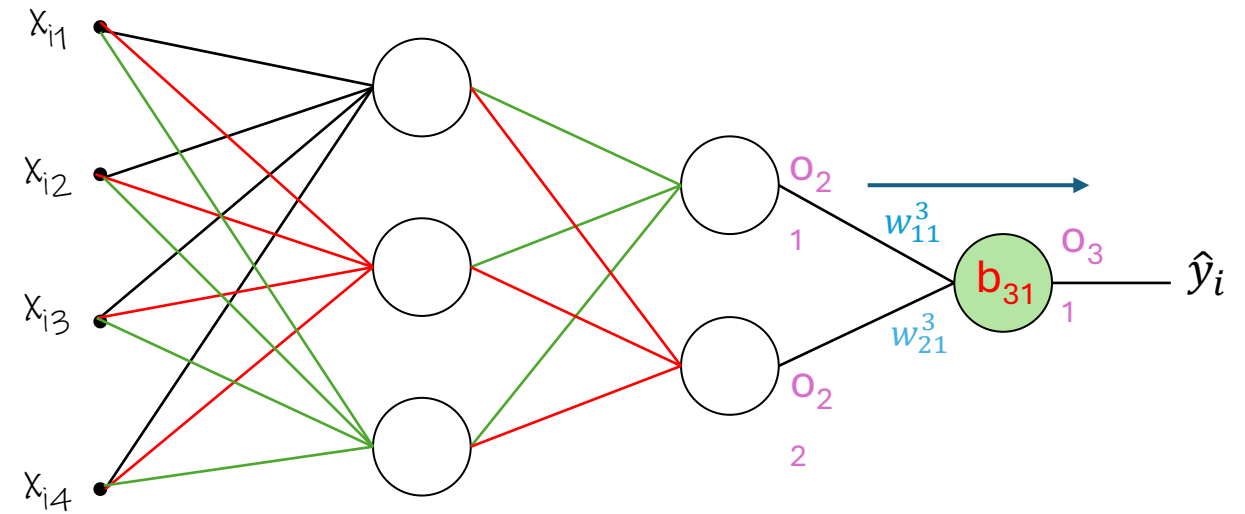


$$\sigma \left(\begin{bmatrix} z_{21} \\ z_{22} \end{bmatrix} \right) = \begin{bmatrix} o_{21} \\ o_{22} \end{bmatrix}$$



Forward Propagation: layer 2

$$z_{31} = w_{11}^3 o_{21} + w_{21}^3 o_{22} + b_{31}$$
$$\hat{y}_i = o_{31} = \sigma(z_{31})$$



Fortunately, this can be done easily using matrix calculation

$$\begin{bmatrix} w_{11}^3 \\ w_{21}^3 \end{bmatrix}^T \cdot \begin{bmatrix} o_{21} \\ o_{22} \end{bmatrix} + b_{31} = z_{31} \quad \longrightarrow \quad \sigma(z_{31}) = o_{31} = \hat{y}_i$$



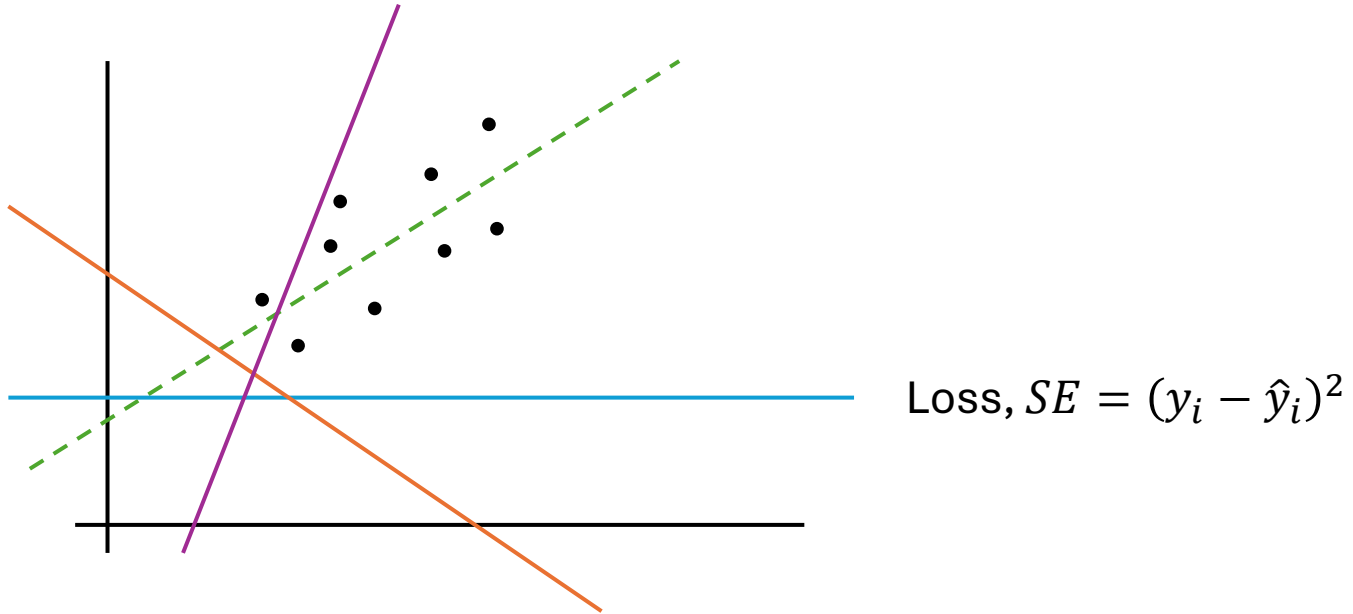
Loss Functions in deep learning

What is Loss Function?

- A method of evaluating how well your algorithm is modelling your dataset.
- Usually high loss $\rightarrow$ poor performance
- Low loss $\rightarrow$ great performance
- In ML/DL, loss is a function of the model parameters.
- Why is loss function important?



Loss function in ML (Linear Regression)



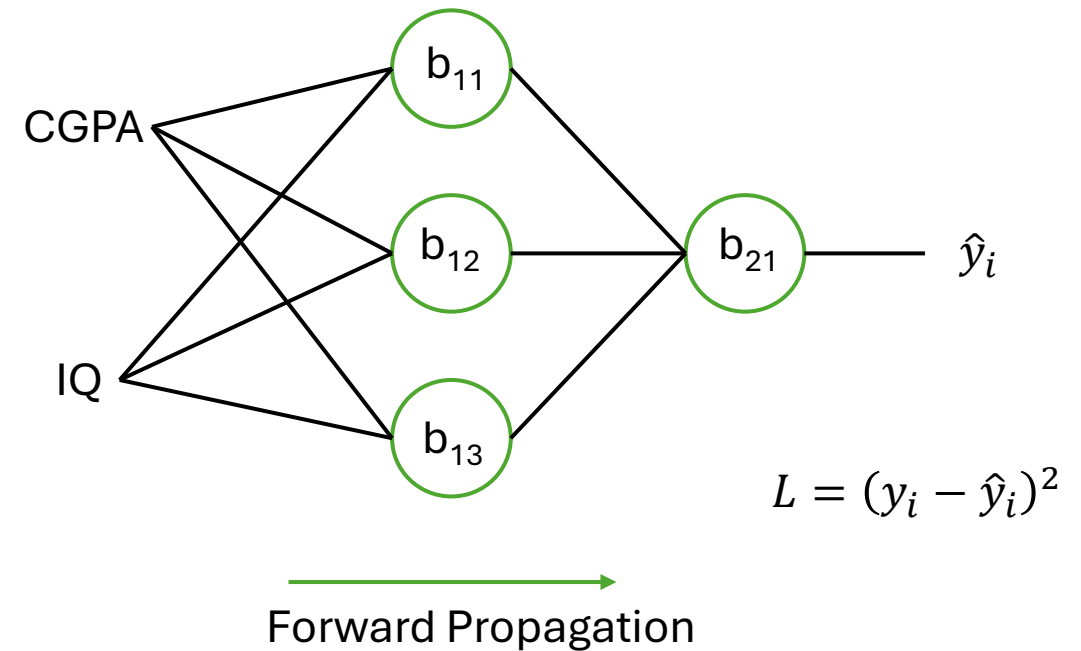
Goal: Find the line that minimizes the “Loss”

Loss function in Deep Learning

CGPA	IQ	Package (Lakh)
7.1	83	3.2
8.5	91	4.5
6.3	102	6.1
5.1	87	2.7
...	...	...
...	...	...
...	...	...

3.7

4.3

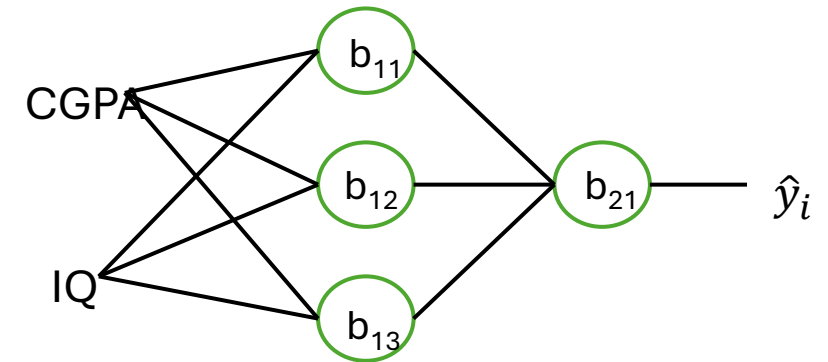


- The weights and biases are initialized randomly.
- After every iteration, these parameters are “adjusted” to pull down the loss.

Loss function vs Cost function

- Loss function: Error in single training
- Cost function: Error in the entire batch

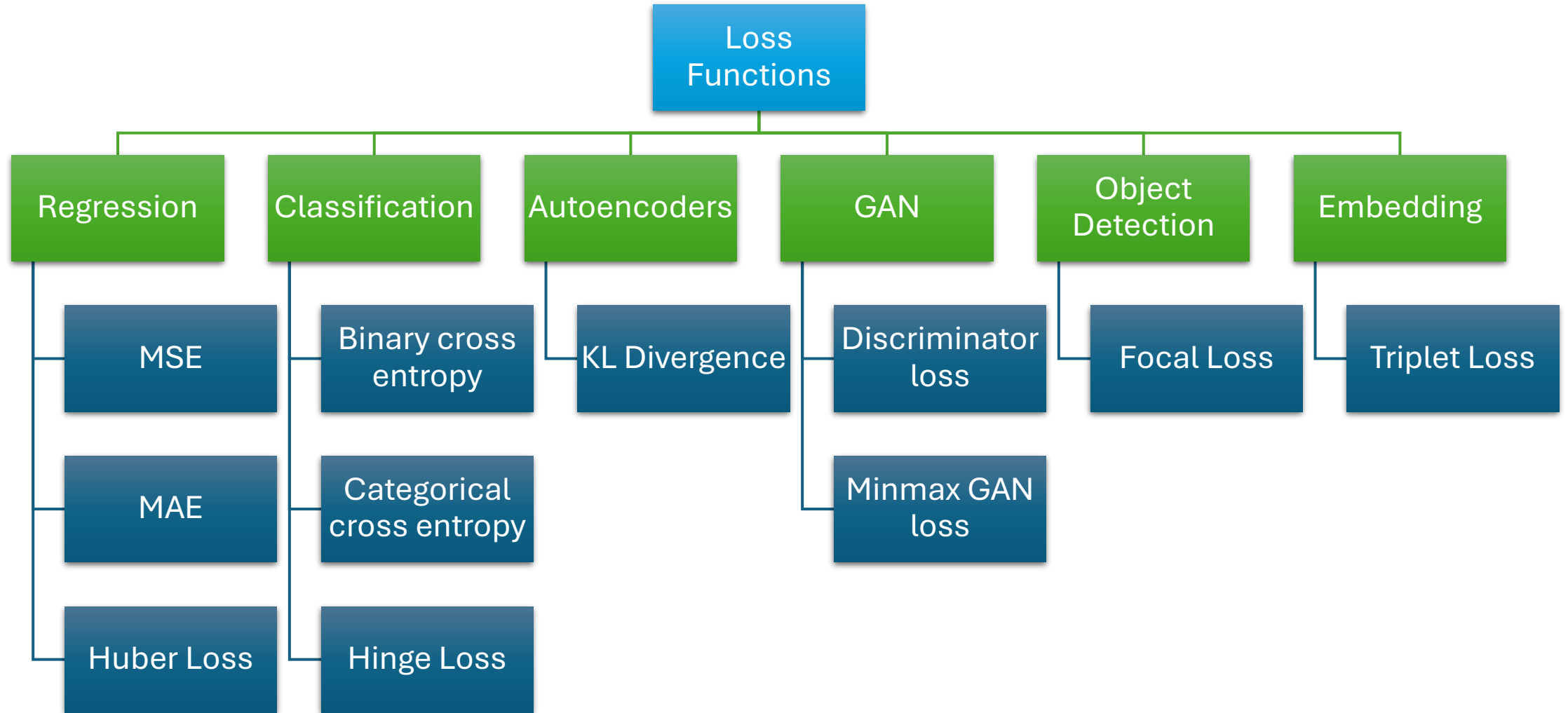
CGPA	IQ	Package (Lakh)	Prediction
6.3	100	6.3	6.1
7.1	91	4.1	4
8.5	83	3.5	3.7
9.2	102	7.2	7



$$L = (y_i - \hat{y}_i)^2 = (6.3 - 6.1)^2$$

$$\text{Cost function} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Various loss functions in DL



Mean Squared Error (MSE)

- Commonly used loss function for **regression** problems.
- It measures the average of the squares of the differences between the actual (true) values and the predicted values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- n is the number of data points,
- y_i is the actual value,
- $\hat{y}_i$ is the predicted value.

MSE (Key characteristics)

- **Squaring the Errors:** By squaring the differences, MSE penalizes larger errors more heavily than smaller ones. This makes MSE highly sensitive to outliers.
- **Units:** The output unit of MSE is the **square of the unit** of the target variable. For example, if the target is measured in dollars, the MSE is in dollars squared.
- **Differentiability:** MSE is a **differentiable** function, which makes it suitable for optimization using gradient-based methods such as stochastic gradient descent (SGD).
- **Convexity:** For linear models, the MSE loss surface is convex, ensuring that gradient descent converges to a global minimum.

Mean Average Error (MAE)

- Another commonly used loss function for **regression** problems.
- It measures the average magnitude of the absolute differences between predicted values and actual values, **without considering their direction** (i.e., it does not square the errors).

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where:

- n is the number of data points,
- y_i is the actual value,
- $\hat{y}_i$ is the predicted value.

MAE (Key characteristics)

- **Linear Penalty:** MAE penalizes all errors in **direct proportion** to their magnitude. This means that **each unit of error contributes equally**, regardless of whether the error is large or small.
- **Robustness to Outliers:** Because it does not square the errors, MAE is **less sensitive to outliers** than MSE. Outliers contribute linearly to the total error.
- **Units:** The output unit of MAE is the **same as the target variable** (e.g., dollars, temperature), making it more interpretable in some real-world contexts.
- **Non-Differentiability at Zero:** MAE is **not differentiable at** $y_i = \hat{y}_i$ due to the sharp corner at zero in the absolute value function. This can make gradient-based optimization less smooth or slower to converge, though subgradient methods are often used in practice.
- **Convexity:** The MAE loss function is **convex**, which guarantees that local minima are also global minima (though the optimization surface is not smooth).

Huber Loss

- **Huber Loss** is a loss function used in regression that combines the advantages of **MSE** and **MAE**.
- It is **quadratic for small errors** and **linear for large errors**, which makes it **less sensitive to outliers** than MSE, while still maintaining differentiability.

- The Huber loss L_δ is defined as:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{for } |y - \hat{y}| \leq \delta \\ \delta \cdot \left(|y - \hat{y}| - \frac{1}{2}\delta \right) & \text{for } |y - \hat{y}| > \delta \end{cases}$$

where:

- The **Huber Loss (average over n samples)** is:

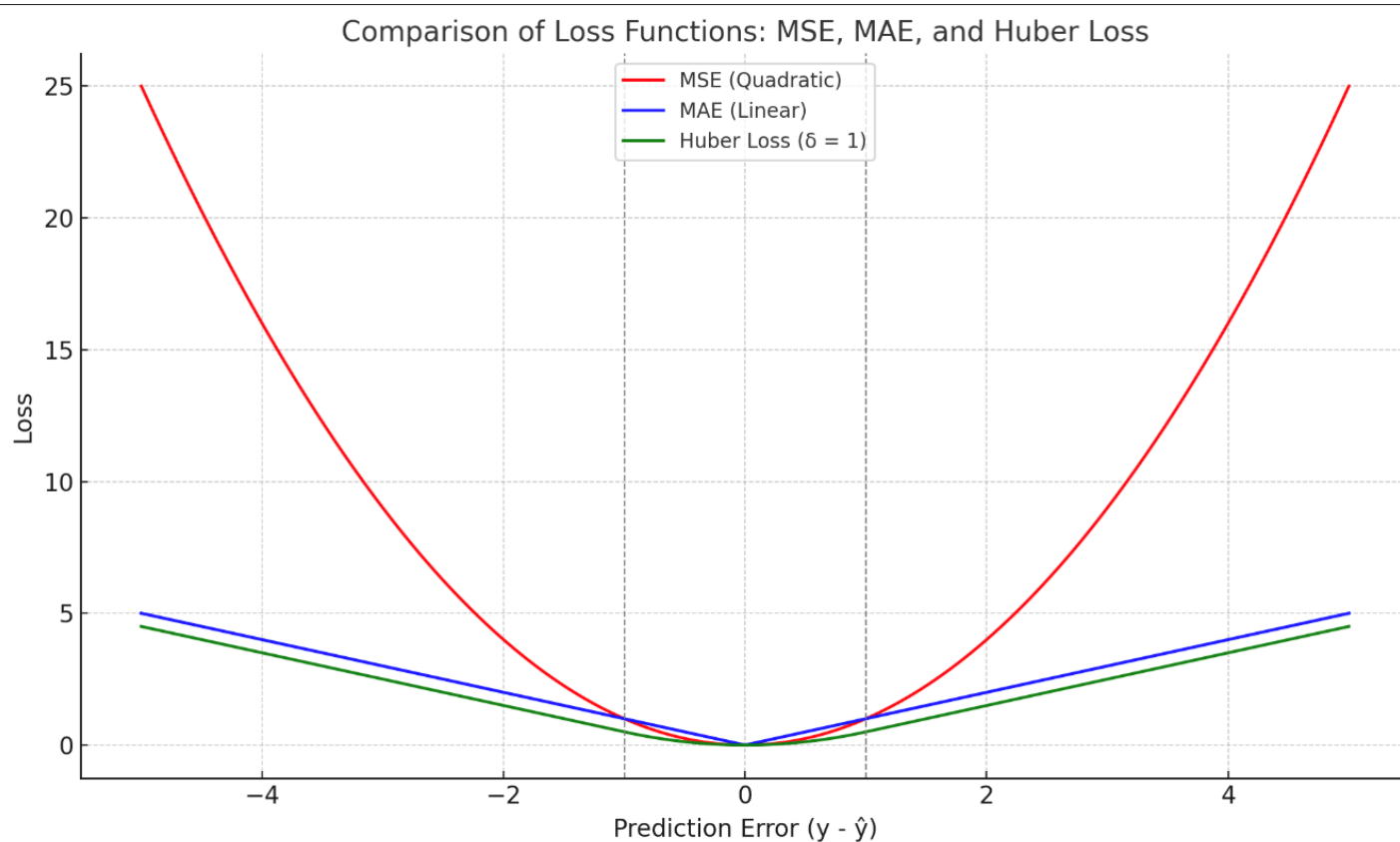
$$\text{Huber}_\delta = \frac{1}{n} \sum_{i=1}^n L_\delta(y_i, \hat{y}_i)$$

- y is the true value;
- $\hat{y}$ is the predicted value;
- δ is a hyperparameter that controls the threshold between the quadratic and linear behavior.

Huber Loss (Key Characteristics)

- **Hybrid Nature:**
 - For small errors ($|y - \hat{y}| \leq \delta$), it behaves like **MSE** (quadratic), ensuring smooth gradients.
 - For large errors ($|y - \hat{y}| > \delta$), it behaves like **MAE** (linear), reducing the influence of outliers.
- **Robustness:** Less sensitive to outliers compared to MSE, which is beneficial in datasets with noisy labels or anomalies.
- **Differentiability:** Fully differentiable everywhere, making it suitable for optimization using gradient-based methods.
- **Parameter δ :** The value of δ must be chosen (or tuned) carefully; smaller values make the loss more robust, but too small a value may under-penalize moderate errors.

Comparison



- **MSE** (red curve): Quadratic growth; penalizes large errors heavily.
- **MAE** (blue curve): Linear growth; treats all errors equally regardless of size.
- **Huber Loss** (green curve): Quadratic for small errors and linear beyond a threshold $\delta = 1$; balances smooth optimization with robustness to outliers.

Summary Comparison

Property	MSE	MAE	Huber Loss
Error Weighting	Squared (emphasizes large errors)	Linear (equal weight to all errors)	Quadratic for small, linear for large
Sensitivity to Outliers	High	Low	Medium
Differentiable	Yes	Not at zero	Yes
Unit of Output	Squared unit of target	Same as target	Depends on regime (quadratic/linear)
Use Case	Smooth optimization, no outliers	Outlier-resistant tasks	Balance between MSE and MAE

Binary Cross Entropy (BCE)

- **Binary Cross-Entropy (BCE)**, also known as **log loss**, is the standard loss function used for binary classification tasks.
- It quantifies the dissimilarity between the **predicted probability** and the **actual class label** (which is either 0 or 1), using the concept of information entropy.
- The **Binary Cross-Entropy Loss** for a single data point is defined as:

$$BCE(y, \hat{y}) = -y \cdot \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

where:

- $y \in \{0, 1\}$ is the true label,
 - $\hat{y} \in \{0, 1\}$ is the predicted probability of the positive class (after applying the sigmoid function),
 - Logarithms are typically natural logs (base e).
- The **mean BCE over n samples** is:

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

$$BCE = \frac{1}{n} \sum_{i=1}^n -y_i \cdot \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)$$

BCE (Key characteristics)

- **Probabilistic Interpretation:**

- BCE models the output as a probability and measures how close the predicted probability $\hat{y}$ is to the actual label y .
- When the model is confident and correct, the loss is low; if confident and wrong, the loss is very high (logarithmic penalty).

- **Asymptotic Behavior:**

- As $\hat{y} \rightarrow y$, the loss approaches zero.
- As $\hat{y} \rightarrow 0$ when $y = 1$, or $\hat{y} \rightarrow 1$ when $y = 0$, the loss approaches infinity.

- **Differentiability:**

- BCE is **fully differentiable**, making it suitable for gradient-based optimization.
- Typically combined with the **sigmoid activation function** in the final layer of the model to map outputs to $[0, 1]$.

BCE (Key characteristics) contd.

- **Convexity:**
 - BCE is **convex** with respect to $\hat{y}$ when y is fixed, which helps ensure stable convergence during training for logistic regression and shallow networks.
- **Output Range:**
 - BCE loss values are **non-negative** and unbounded above.
- **Sensitivity:**
 - BCE is **very sensitive to confident misclassifications** — if the model predicts a probability close to 0 for a true class 1, the loss becomes very large.

Common Use Case:

Binary classification tasks such as spam detection, medical diagnosis (disease vs. no disease), and binary sentiment analysis (positive vs. negative).

Categorical Cross Entropy (CCE)

- **Categorical Cross-Entropy (CCE)** is a loss function used in **multi-class classification** tasks, where each input belongs to exactly one of K possible classes.
- It measures the dissimilarity between the **true class distribution** (usually one-hot encoded) and the **predicted class probability distribution**.
- The **CCE loss for a single sample** is defined as:

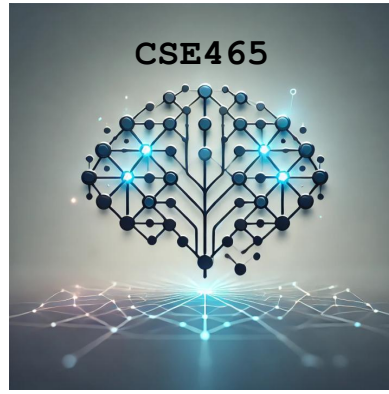
$$CCE(y, \hat{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

Where:

- K is the number of classes,
- $y_k \in \{0, 1\}$ is the indicator for the true class (i.e., 1 for the correct class, 0 otherwise),

- The **average CCE over n samples** is:

$$CCE = \frac{1}{n} \sum_{i=1}^n \left[- \sum_{k=1}^K y_k \log \hat{y}_k \right]$$



Gradient Descent

Linear Regression

Exp, x_1	Profile, x_2	Gender, x_3	Salary, y
7.5	Web	M	2.1 L
2.5	Android	M	60 K
6	ML	F	1.8 L
8	Web	F	2.5 L
5	ML	F	80 K

4D

2D $\rightarrow y = mx + b \rightarrow$ Line

3D $\rightarrow y = ax_1 + bx_2 + c \rightarrow$ Plane

4D $\rightarrow y = ax_1 + bx_2 + cx_3 + d \rightarrow$ Hyperplane

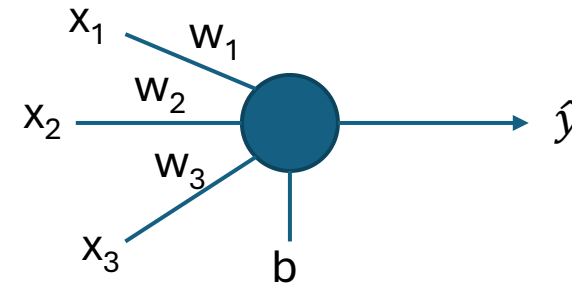
a, b, c

weights

d

Bias (offset)

Very close to neural networks



What Is Optimization?

- **Optimization** is the process of **adjusting the model parameters to minimize the loss function**.
- It's like finding the bottom of a valley in a loss landscape — the lowest possible value of the loss based on the current weights and biases.
- Imagine the loss function as a surface or curve:
 - The **x and y axes** are the weights (W) and biases (b)
 - The **z axis** is the loss value
- Optimization is the process of **navigating this surface** to find the **lowest point (minimum)** — the point where the model performs best.
- How Do We Minimize the Loss?
 - This is where **optimization algorithms** come in.
 - The most common one in neural networks is: **Gradient Descent**

Gradient Descent

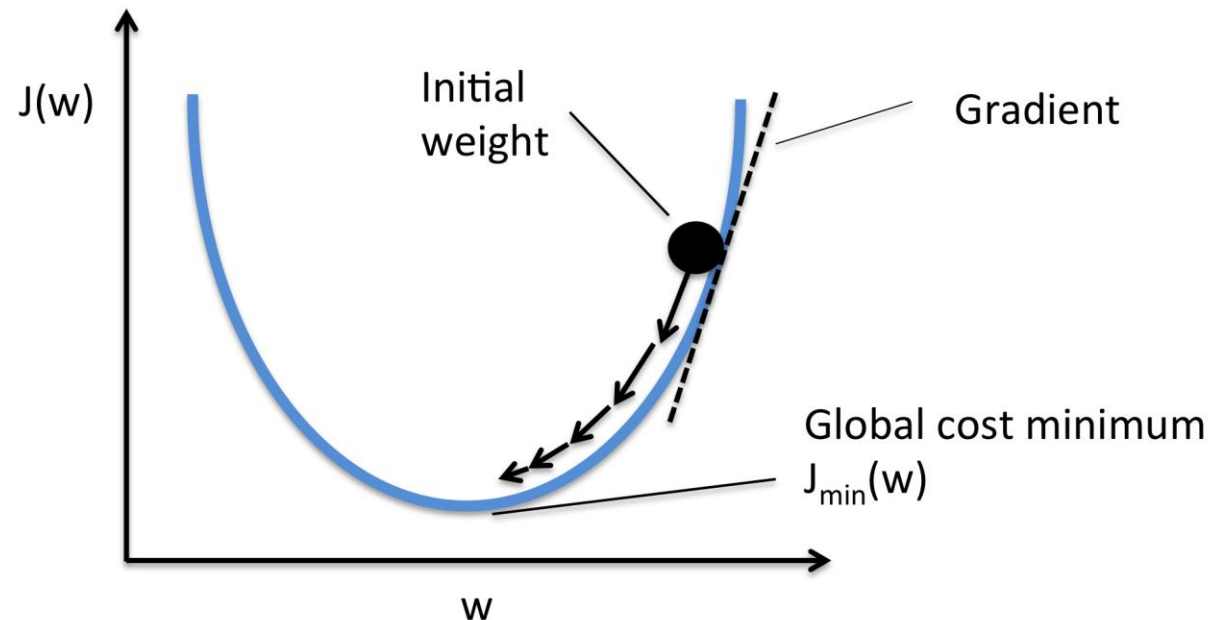
- Applied in:
 - Linear Regression, Logistic Regression, PCA, Neural Network (Backbone of DL)
- **Gradient Descent** is an optimization algorithm used to **minimize a cost function** (also called the loss function) by iteratively moving in the direction of the **steepest descent**, i.e., the **negative of the gradient**.
- Analogy: Hiking down a hill blindfolded

Imagine you're standing on a hilltop, blindfolded. Your goal is to reach the lowest point in the valley.

 - You can't see where to go. So, you feel the ground around you to check which direction is sloping down.
 - You then take a small step in that direction.
 - Repeat this process until you can't go any lower – voila!! The bottom!!!

How it works: Simple terms

- A model makes a prediction
- It checks how wrong the prediction is (using a loss function)
- Then it calculates which direction to change the parameters to reduce the error (gradient).
- It takes a small step in that direction (descent).
- Repeat this over and over until the error is as small as possible.



Mathematical Formulation

Let:

- $\mathcal{L}(\theta)$: loss function
- θ : parameter vector (weights and biases in a neural network)
- η : learning rate (step size)

Update Rule:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta)$$

Repeat this step until the loss converges or a stopping condition is met.

How Gradient Descent Works in Neural Networks

In a neural network, θ includes:

- All weights $W^{[l]}$ and biases $b^{[l]}$ for each layer l .

At each iteration (epoch):

1. Do a **forward pass** to compute the prediction $\hat{y}$
2. Compute the **loss** between $\hat{y}$ and actual y
3. Do **backward pass** (backpropagation) to compute $\frac{\partial \mathcal{L}}{\partial \theta}$
4. Update parameters using gradient descent

Variants of Gradient Descent

- The **key difference** in gradient descent variants lies in **how much data** is used to compute the gradient at each step.
 - 1) Batch Gradient Descent (Vanilla Gradient Descent)
 - 2) Stochastic Gradient Descent (SGD)
 - 3) Mini-Batch Gradient Descent (Most Common in Practice)

1. Batch Gradient Descent (Vanilla GD)

- Uses **all training samples** to compute the gradient
- Very **accurate gradient**, but **computationally expensive** for large datasets
- Update rule:

$$\theta \leftarrow \theta - \eta \cdot \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \mathcal{L}(x^{(i)}, y^{(i)})$$

- m: number of training samples

Pros:

- Stable convergence
- Fewer oscillations

Cons:

- Slow for large datasets
- Memory intensive

2. Stochastic Gradient Descent (SGD)

- Uses **one training example** per update
- Fast, but **noisy estimates of the gradient**
- Update rule:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(x^{(i)}, y^{(i)})$$

Pros:

- Very fast
- Can escape local minima due to noise

Cons:

- High variance in updates
- May not converge smoothly

3. Mini-Batch Gradient Descent

- Uses a **small batch** (e.g., 32, 64, 128 samples) to compute each gradient
- Balance between speed and stability
- Update rule:

$$\theta \leftarrow \theta - \eta \cdot \frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \mathcal{L}(x^{(i)}, y^{(i)})$$

- k: mini-batch size (e.g., 32)

Pros:

- Fast and efficient
- Parallelizable
- Smooth convergence

Cons:

- Still a bit noisy
- Needs tuning of batch size

Choosing the Learning Rate (η)

- This is a hyperparameter.
- Too **small**: Convergence is slow
- Too **large**: May overshoot or diverge
- Use **learning rate schedulers** to adjust over time

Advanced Optimizers (Beyond Basic GD)

While not strictly "variations" of gradient descent, these are **improvements based on GD**:

Optimizer	Key Feature
Momentum	Accelerates GD by adding velocity
RMSProp	Adapts learning rate per parameter
Adam	Combines momentum and RMSProp
Adagrad	Increases learning rate for infrequent parameters

Additional Concepts

a) **Epoch vs Iteration:**

- **Epoch** = 1 pass through full dataset
- **Iteration** = 1 update step (i.e., per batch or mini-batch)

b) **Loss Landscape:**

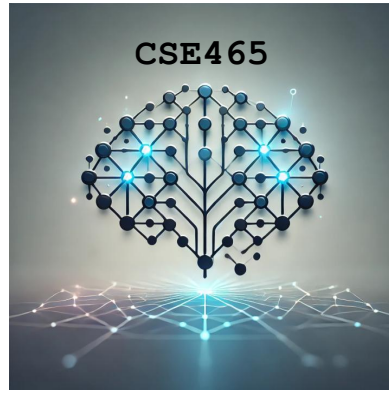
- The shape of the cost function affects how gradients behave
- Flat areas → slow learning; steep areas → possible instability

c) **Convergence Criteria:**

- Early stopping (based on validation loss)
- Max number of epochs
- Gradient norm approaching zero

Summary

Variant	Data Used	Speed	Stability	Use Case
Batch GD	All data	Slow	Stable	Small datasets
Stochastic GD	One sample	Fast	Unstable	Online learning
Mini-batch GD	Small batch	Balanced	Balanced	Deep learning (✓)



Backpropagation: A complete guide

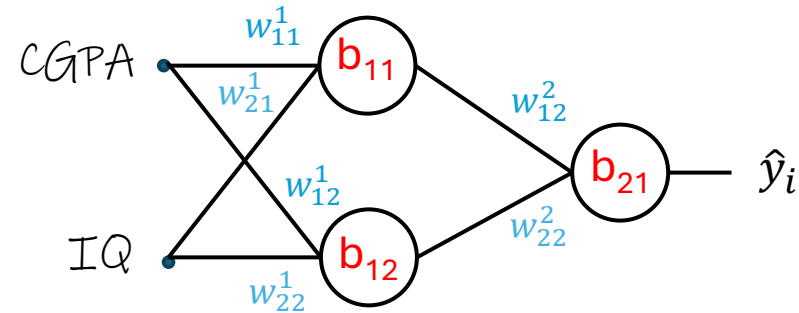
The “What”? - Understanding backpropagation

- Short for “backward propagation of error”.
- The derivation of the backpropagation algorithm as used and understood in a modern machine learning context was first published by David E. Rumelhart, Geoffrey Hinton and Ronald J. Williams in their 1986 paper, “Learning representations by back-propagating errors”. [2]
- Backpropagation is a machine learning technique essential to the optimization of artificial neural networks. It facilitates the use of [gradient descent](#) algorithms to update network weights, which is how the deep learning models driving modern artificial intelligence (AI) “learn.” [3]
- Prerequisites for Understanding Backpropagation:
 - Gradient Descent: An optimization algorithm
 - Forward Propagation: A technique for neural network predictions

Purpose of Backpropagation Algorithm

- An algorithm used to **train** Neural Networks
- Example:

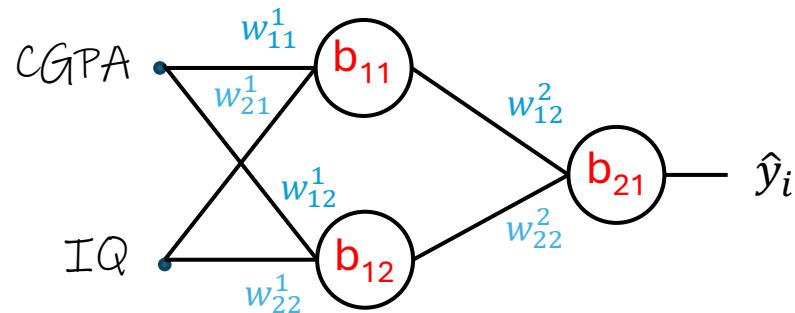
CGPA	IQ	Package (Lakh)
7.1	83	3.2
8.5	91	4.5
6.3	102	6.1
5.1	87	2.7



- To find the **optimum values** for **weights** and **biases** for a given dataset, enabling the neural network to make the best predictions

The Backpropagation Algorithm: A Series of Steps

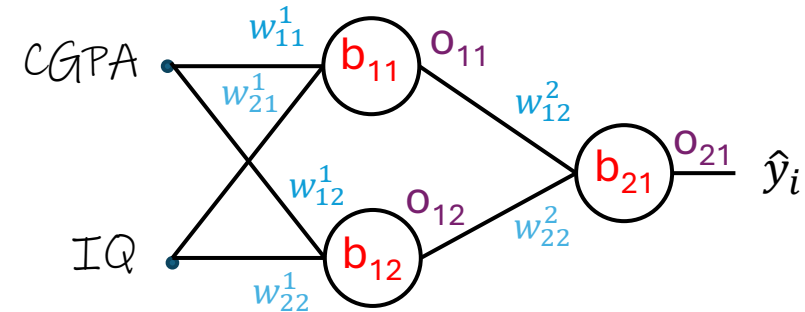
CGPA	IQ	Package (Lakh)
7.1	83	3.2
8.5	91	4.5
6.3	102	6.1
5.1	87	2.7



- Step 0: Initialize Weights and Biases
 - Methods: Random values, or specific values like weights = 1 and biases = 0.
- Step 1: Select a Data Point
 - Choose one row of data from the dataset (e.g., a student's CGPA and IQ)
- Step 2: Forward Propagation to Get a Prediction, $\hat{y}_i$
 - Feed the selected data point into the neural network.
 - Calculate the output (prediction) using the current weights and biases
- Step 3: Calculate the Loss (Error)
 - Use a suitable **Loss Function** (e.g., MSE for regression problems)
- Step 4: Update Weights and Biases
 - Adjust weights and biases to reduce the calculated loss. This is where **Backpropagation** happens – propagating the error backward through the network

Backpropagation Example: Regression

CGPA	IQ	Package (Lakh)
7.1	83	3.2
8.5	91	4.5



- Assumption: For simplicity, no activation in any neurons.
- Loss: $MSE = (y_i - \hat{y}_i)^2$
- Target: Minimize Loss.
- We can't change y_i since this is ground truth, we can however change $\hat{y}_i$
- $\hat{y}_i = o_{21} = w_{12}^2 o_{11} + w_{22}^2 o_{12} + b_{21}$
- $o_{11} = w_{11}^1 * CGPA + w_{21}^1 * IQ + b_{11}$
- $o_{12} = w_{12}^1 * CGPA + w_{22}^1 * IQ + b_{12}$

Recap: Chain Rule of Differentiation

- The **chain rule** is a formula used to **differentiate a function of a function**—also called a **composite function**.
- In simple terms: If a function is **nested inside another**, we **differentiate each layer one at a time**, from the outside in, and multiply the results.
- Mathematical Form (Single Variable):

- Let, $y = f(g(x))$.
- Then the derivative is:

$$\frac{dy}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

- Or written with function names:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

Recap: Partial Derivatives

- When a function depends on **more than one variable**, a **partial derivative** tells us how the function changes when **only one variable changes**, while the **others are kept constant**.

- Mathematical Definition:

- Let $f(x,y)$ be a function of two variables.

- **Partial derivative with respect to x is:**

$$\frac{\partial f}{\partial x} = \text{change in } f \text{ as } x \text{ changes, keeping } y \text{ constant}$$

- **Partial derivative with respect to y is:**

$$\frac{\partial f}{\partial y} = \text{change in } f \text{ as } y \text{ changes, keeping } x \text{ constant}$$

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

Recap: Gradient Descent Update Rule

- Weights and bias update:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}}$$

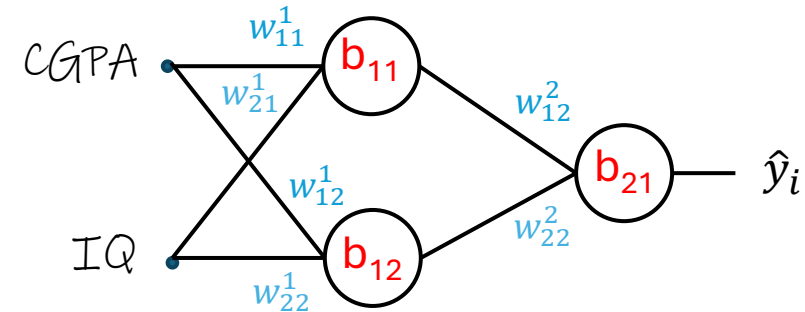
$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b_{old}}$$

- This rule is applied for all trainable parameters (weights and biases)

Calculation of Derivatives: Output Layer

Backpropagation Example: Classification

CGPA	IQ	Placement
8	83	1
7	91	1
6	100	0
5	59	0



- 9 trainable parameters
- Activation functions: Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Loss (Binary Cross Entropy) = $-y \log \hat{y} - (1 - y) * \log(1 - \hat{y})$
- $\sigma'(z) = \sigma(z) * (1 - \sigma(z))$

Calculation of Derivatives: Output Layer

Let's assume:

- The output layer is: $\hat{y} = \sigma(z)$
- The loss is BCE applied to $\hat{y}$.
- Using the chain rule:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

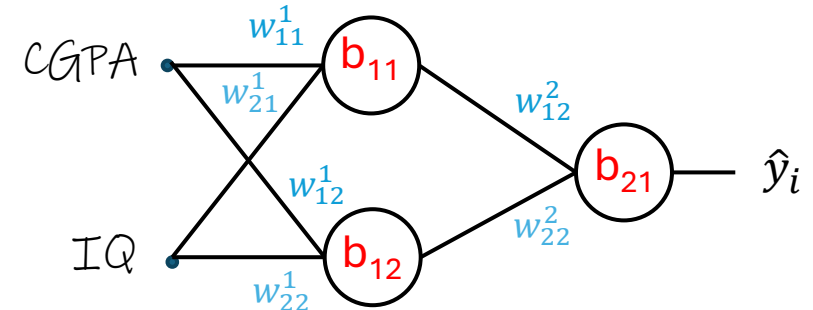
- Recall:

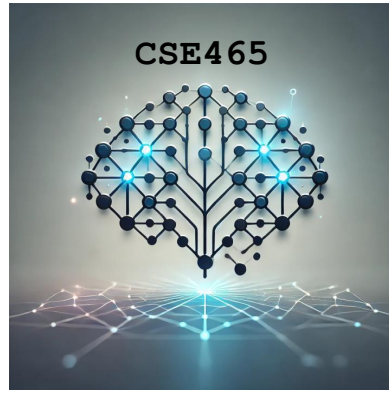
$$\frac{\partial \hat{y}}{\partial z} = \frac{\partial}{\partial z} \sigma(z) = \sigma(z) * (1 - \sigma(z)) = \hat{y}(1 - \hat{y})$$

$$\frac{\partial L}{\partial \hat{y}} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right)$$

- Which simplifies to:

$$\frac{\partial L}{\partial z} = \hat{y} - y$$





Intermediate Value Caching

Why Is Intermediate Value Caching Necessary?

- Training an artificial neural network (a.k.a. MLP) involves **two computationally expensive processes**:

a) Forward Pass:

At each layer, we compute:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \text{ (Linear transformation)}$$

$$a^{[l]} = \text{Activation}(z^{[l]}) \text{ (Non-linearity)}$$

b) Backward Pass (Backpropagation):

Uses the **chain rule of calculus** to compute gradients:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}}, \frac{\partial \mathcal{L}}{\partial b^{[l]}}$$

These gradients are used to **update weights** via gradient descent.

Problem:

- To compute gradients during backpropagation, we need many intermediate values (like $z^{[l]}$, $a^{[l]}$).
- If we don't store these during the forward pass, we'd have to **recalculate them** — this is:
 - Computationally expensive
 - Wasteful, especially for deep networks

Solution: Cache intermediate values during forward pass.

What Is Intermediate Value Caching?

- *Intermediate value caching* is the process of **storing intermediate computations** (like linear outputs and activations) during the forward pass, so they can be **reused during the backward pass** without re-computation.
- What Is Cached?

During the **forward pass**, for each layer ℓ , we typically cache:

Symbol	Meaning	Purpose
$a^{[l-1]}$	Input activation to layer	Needed to compute $\frac{\partial \mathcal{L}}{\partial W^{[l]}}$
$z^{[l]}$	Linear output $W^{[l]}a^{[l-1]} + b^{[l]}$	Needed for derivative of activation
$a^{[l]}$	Activated output	Often reused in next layer or loss

- In frameworks like **PyTorch** and **TensorFlow**, this caching happens **automatically** using a **computation graph or gradient tape**.



Vanishing/Exploding Gradient

Motivation: Why Gradients Matter

- In training neural networks, we use **gradient-based optimization** (like gradient descent) to update weights and biases so that the model improves over time.
- For each parameter θ , we update:

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

- η : learning rate
- $\frac{\partial \mathcal{L}}{\partial \theta}$: gradient of loss with respect to that parameter
- To compute these gradients efficiently, we use the **backpropagation algorithm** and the **chain rule** of calculus.

The Chain Rule in Backpropagation

- Consider a deep network with many layers. Let's say we want to compute the gradient of the loss with respect to some earlier layer's weights.
- We apply the **chain rule**:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{\partial \mathcal{L}}{\partial a^{[n]}} \cdot \frac{\partial a^{[n]}}{\partial z^{[n]}} \cdots \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial W^{[1]}}$$

- Each term is a **derivative**, often involving small numbers (e.g., from sigmoid or tanh activations).

What Is the Vanishing Gradient Problem?

- **Vanishing Gradient Problem** occurs when the gradients (derivatives) become extremely small as we move backward through layers of a deep network. This leads to **very slow learning or no learning** in the early layers of the network.
- Intuition:
 - Suppose each derivative is ≤ 0.5 .
 - Multiplying many of them together causes the overall gradient to **shrink exponentially**.

If $\delta = 0.5^n$, then for $n = 10$, $\delta = 0.00098$

- This small gradient **barely updates the weights** of early layers.
- The network gets stuck — it doesn't learn deep features.

Why Does It Happen? (Root Causes)

a) Sigmoid or Tanh Activation Functions:

- Sigmoid range: $(0, 1)$; tanh range: $(-1, 1)$
- Their derivatives:
 - Sigmoid derivative: $\sigma'(z) = \sigma(z)(1-\sigma(z)) \rightarrow \text{max value} \approx 0.25$
 - Tanh derivative: $1 - \tanh^2(z) \rightarrow \text{also} < 1$
- So repeated multiplication of small derivatives causes the gradient to **vanish**.

b) Deep Networks:

- The deeper the network, the more times small values are multiplied.
- Gradient **shrinks** before it reaches early layers.

What Happens When Gradients Vanish?

- **Early layers** barely learn (no significant weight update).
- **Slow convergence** or **no convergence**.
- Model performs poorly despite training for a long time.
- Especially problematic in:
 - Deep MLPs
 - RNNs (Recurrent Neural Networks)

Solutions to Vanishing Gradient

Let's now explore both **practical solutions** and **architectural innovations**:

- a) Use ReLU Activation Instead of Sigmoid/Tanh
- b) Proper Weight Initialization
- c) Batch Normalization
- d) Residual Connections (Skip Connections)
- e) Use Architectures Less Prone to This

Reference and further reading

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
2. Rumelhart, D., Hinton, G. & Williams, R. Learning representations by back-propagating errors. *Nature* **323**, 533–536 (1986). <https://doi.org/10.1038/323533a0>
3. D. Bergmann, C. Stryker, “What is Backpropagation?”, IBM Think, [Online]. Available: <https://www.ibm.com/think/topics/backpropagation>. [Accessed: Jun. 23, 2025].
4. Pramoditha, Rukshan. “The Concept of Artificial Neurons (Perceptrons) in Neural Networks.” *Medium*, Towards Data Science, 29 Dec. 2021.[Online]. Available: towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc. [Accessed 21 Jan. 2025].